The Transformer

A Complete Guide — From Tokenization to Vision Transformers, Multimodal Models, and Robotics

Prepared for Sunkeerth

August 2026

Contents

1 Chapter 1 — Introduction: Why the Transformer Changed Everything 4
- 1.1 The One-Sentence Idea . . . 4
- 1.2 An Analogy: Reading a Sentence vs. Reading Word by Word Through a Straw . . . 4
- 1.3 Why This Mattered Enough to Replace RNNs Almost Overnight . . . 4
- 1.4 A Five-Second History . . . 4
- 1.5 What This Book Covers, In Order . . . 5

2 Chapter 2 — Tokenization: Turning Text Into Numbers 6
- 2.1 Why This Step Exists at All . . . 6
- 2.2 Three Levels of Tokenization, and Why Subwords Won . . . 6
- 2.3 How Subword Vocabularies Are Actually Built . . . 6
- 2.4 Token IDs and the Vocabulary . . . 7
- 2.5 Special Tokens . . . 7
- 2.6 A Worked Example, Traced by Hand . . . 7
- 2.7 Why This Chapter Matters More Than It Seems . . . 7

3 Chapter 3 — Embeddings: Giving Each Token a Meaning 9
- 3.1 From Integer to Vector . . . 9
- 3.2 The Analogy: A Map of Meaning . . . 9
- 3.3 Embedding Dimensionality . . . 9
- 3.4 Weight Tying: Reusing the Embedding Matrix . . . 9
- 3.5 A Note on Vision and Other Modalities . . . 11

4 Chapter 4 — Positional Encoding: Teaching the Model About Order 12
- 4.1 The Problem This Solves . . . 12
- 4.2 Sinusoidal Positional Encoding (the Original Approach) . . . 12
- 4.3 Learned Positional Embeddings . . . 13
- 4.4 Modern Approaches: RoPE and ALiBi . . . 13
- 4.5 Why This Chapter Is Short But Not Small . . . 13

5 Chapter 5 — Self-Attention: The Heart of the Transformer 14
- 5.1 The Intuition First, Before Any Math . . . 14
- 5.2 Query, Key, Value — A Second Analogy: A Library Search . . . 14
- 5.3 The Formula, Explained Term by Term . . . 15
- 5.4 A Fully Worked Numeric Example . . . 15
- 5.5 Why “Self” Attention, Specifically . . . 17

6 Chapter 6 — Multi-Head Attention: Several Perspectives at Once 18
6.1 The Limitation of a Single Attention Computation . . . 18
6.2 The Fix: Run Attention Multiple Times, in Parallel, in Smaller Subspaces . . . 18
6.3 What Different Heads Actually Learn — Documented, Interpretable Patterns . . . 19
6.4 Practical Sizing . . . 19

7 Chapter 7 — The Full Transformer Block 20
7.1 Assembling the Pieces . . . 20
7.2 The Two Sublayers . . . 20
7.3 Why the Residual Connections Are Not Optional . . . 21
7.4 Why Layer Normalization, Specifically (Not Batch Normalization) . . . 21
7.5 Stacking Blocks . . . 22

8 Chapter 8 — Three Flavors: Encoder-Only, Decoder-Only, Encoder-Decoder 23
8.1 Why the Same Building Block Splits Into Three Architectures . . . 23
8.2 Encoder-Only (BERT-Style): Full, Bidirectional Attention . . . 23
8.3 Decoder-Only (GPT-Style): Causal (Masked) Attention . . . 24
8.4 Encoder-Decoder (T5-Style, and the Original 2017 Architecture) . . . 24
8.5 Choosing Among the Three, in Practice . . . 25

9 Chapter 9 — How Transformers Are Actually Trained 26
9.1 Pretraining vs. Fine-Tuning: The Two-Stage Recipe . . . 26
9.2 Instruction Tuning and Alignment (Brief, for Context) . . . 26
9.3 Teacher Forcing (Inherited Directly From the RNN World) . . . 26
9.4 The Loss Function . . . 26
9.5 Training at the Scale Modern Transformers Require . . . 26
9.6 A Note on Data Quality and Scale . . . 27

10 Chapter 10 — A Survey of Transformer Types 28
10.1 Text-Only Transformers . . . 28
10.2 Vision Transformers (Full Deep Dive in Chapter 11) . . . 28
10.3 Multimodal Transformers (Full Discussion in Chapter 12) . . . 28
10.4 Robotics-Oriented Transformers (Full Discussion in Chapter 15) . . . 29
10.5 Efficient / Long-Context Transformers (Full Discussion in Chapter 13) . . . 29
10.6 The Common Thread . . . 29

11 Chapter 11 — Vision Transformer (ViT): Deep Dive 30
11.1 The Provocative Original Claim . . . 30
11.2 Step 1 — Splitting the Image Into Patches . . . 30
11.3 Step 2 — Linear Projection: The "Tokenization" of Vision . . . 30
11.4 Step 3 — The [CLS] Token, Borrowed Directly From BERT . . . 31
11.5 Step 4 — Positional Embeddings . . . 31
11.6 Step 5 — A Completely Standard Transformer Encoder . . . 31
11.7 Step 6 — Classification Head . . . 31
11.8 Why This Actually Works — And Its One Real Weakness . . . 32
11.9 Important ViT Variants . . . 32
11.10 Where ViT Fits in the Bigger Picture of This Book . . . 32

12 Chapter 12 — Multimodal Transformers: Combining Vision and Language 33
12.1 Why This Is Now Straightforward, Given Everything Before This Chapter . . . 33
12.2 CLIP: Learning a Shared Space for Images and Text . . . 33
12.3 Vision-Language Models (VLMs): From Alignment to Generation . . . 34
12.4 Why This Design Is So Reusable . . . 34

13 Chapter 13 — Efficient Attention: Scaling to Long Sequences 35

13.1 The Problem, Stated Precisely . . . 35
13.2 FlashAttention: Same Math, Dramatically Faster . . . 35
13.3 Sparse and Windowed Attention: Genuinely Changing What Gets Computed . . . 35
13.4 Linear Attention Approximations . . . 35
13.5 Mixture of Experts (MoE): A Different Axis of Efficiency Entirely . . . 36
13.6 A Practical Framing . . . 36

14 Chapter 14 — Where Transformers Are Used: An Industry Survey 37
14.1 Natural Language Processing (The Original Home Turf) . . . 37
14.2 Computer Vision . . . 37
14.3 Speech and Audio . . . 37
14.4 Science and Biology . . . 37
14.5 Finance and Time Series . . . 37
14.6 Recommendation Systems . . . 37
14.7 Enterprise and Productivity Software . . . 38
14.8 And, the Focus of the Next Chapter — Robotics . . . 38

15 Chapter 15 — Transformers in Robotics and Embodied AI 39
15.1 The Core Idea: Treat a Robot Action as Just Another Token . . . 39
15.2 Vision-Language-Action (VLA) Models . . . 39
15.3 Concrete Named Systems . . . 40
15.4 How This Connects to Imitation Learning (RL+IL Book, Chapters 12–15) . . . 40
15.5 Why This Matters for ROS 2-Based Systems Specifically . . . 41

16 Chapter 16 — Implementing a Transformer in PyTorch 42
16.1 Tokenization (Chapter 2) — Using a Real Library . . . 42
16.2 Embeddings (Chapter 3) . . . 42
16.3 Positional Encoding (Chapter 4) — Sinusoidal, From Scratch . . . 42
16.4 Scaled Dot-Product Self-Attention (Chapter 5) — From Scratch . . . 42
16.5 Multi-Head Attention (Chapter 6) — From Scratch . . . 43
16.6 A Full Transformer Encoder Block (Chapter 7) — From Scratch . . . 43
16.7 Causal Masking for a Decoder-Only Model (Chapter 8) . . . 44
16.8 A Minimal Vision Transformer Patch Embedding (Chapter 11) . . . 44
16.9 Using a Pretrained Transformer Directly (What You'll Actually Do Most Often) . . . 44
16.10 Closing Note . . . 44

1 Chapter 1 — Introduction: Why the Transformer Changed Everything

1.1 The One-Sentence Idea

A Transformer is a neural network built entirely around one operation — **letting every element of a sequence look at every other element and decide, for itself, what's relevant** — rather than processing the sequence step by step like older architectures did.

1.2 An Analogy: Reading a Sentence vs. Reading Word by Word Through a Straw

Imagine reading the sentence: *"The trophy didn't fit in the suitcase because it was too big."* What does "it" refer to — the trophy or the suitcase? You answer this instantly, by glancing back and forth across the *whole sentence at once*, weighing "trophy" and "suitcase" against "too big" simultaneously.

Now imagine being forced to read this sentence one word at a time through a narrow straw, only ever remembering a compressed summary of everything before the current word, never able to glance back at the exact original words. This is roughly what an RNN (covered in the earlier RNN book) does — it processes tokens strictly in order, carrying forward only a fixed-size memory. A Transformer removes the straw entirely: every word can directly look at every other word, all at once, and decide how much attention to pay to each — exactly the way you resolved "it" a moment ago. This single change — replacing sequential memory with direct, all-at-once lookup — is the entire idea behind why Transformers work as well as they do.

1.3 Why This Mattered Enough to Replace RNNs Almost Overnight

Two decisive advantages, covered in depth throughout this book:

1. **Parallelization.** An RNN must compute step 1 before step 2 before step 3 — inherently sequential, and slow to train on modern GPUs, which are built to do many things at once, not one thing very fast. A Transformer computes attention for every position *simultaneously*, which is a perfect fit for GPU/TPU parallelism, and is the practical reason it became feasible to train models with hundreds of billions of parameters.
2. **Direct long-range access.** In an RNN, information from 50 steps ago has to survive 50 repeated transformations to influence the current step (and, as the RNN book showed in detail, it usually doesn't survive intact). In a Transformer, a token 50 positions away is exactly **one attention step** away — no decay, no bottleneck.

1.4 A Five-Second History

- **2014**: Attention is bolted onto RNN encoder-decoders (seq2seq) to fix a compression bottleneck — covered in the RNN book, Chapter 8.
- **2017**: *"Attention Is All You Need"* (Vaswani et al.) proposes removing the RNN entirely and building a model purely out of attention. This is the Transformer.
- **2018–2019**: BERT (encoder-only, understanding tasks) and GPT (decoder-only, generation) show the architecture scales dramatically with more data and parameters.
- **2020**: Vision Transformer (ViT) shows the same architecture works for images, not just text — no convolutions required at all.
- **2021 onward**: CLIP, multimodal models, and eventually Vision-Language-Action (VLA) models bring the same core architecture into robotics — the same attention mechanism

that resolves “it” in a sentence is, today, deciding which pixels in a camera feed matter for the next motor command.

1.5 What This Book Covers, In Order

Every layer a token passes through, in the order it actually happens: **tokenization** (Ch. 2) → **embeddings** (Ch. 3) → **positional encoding** (Ch. 4) → **self-attention** (Ch. 5) → **multi-head attention** (Ch. 6) → **the full Transformer block** (Ch. 7) → **encoder-only vs. decoder-only vs. encoder-decoder architectures** (Ch. 8) → **how Transformers are trained** (Ch. 9) → a **survey of major Transformer types** (Ch. 10) → a deep dive into the **Vision Transformer** (Ch. 11) → **multimodal models** like CLIP (Ch. 12) → **efficient attention** for scaling to long sequences (Ch. 13) → **industrial applications** across domains (Ch. 14) → a focused chapter on **Transformers in robotics and embodied AI** (Ch. 15) → and finally **working PyTorch code** for everything (Ch. 16).

2 Chapter 2 — Tokenization: Turning Text Into Numbers

2.1 Why This Step Exists at All

Neural networks only understand numbers — vectors of floats being multiplied by weight matrices. A sentence like *"Robots learn quickly"* has to be converted into numbers before any Transformer layer can touch it. **Tokenization** is that conversion, and it happens in two stages: splitting text into pieces (**tokens**), then looking each piece up in a fixed vocabulary to get an integer (**the token ID**).

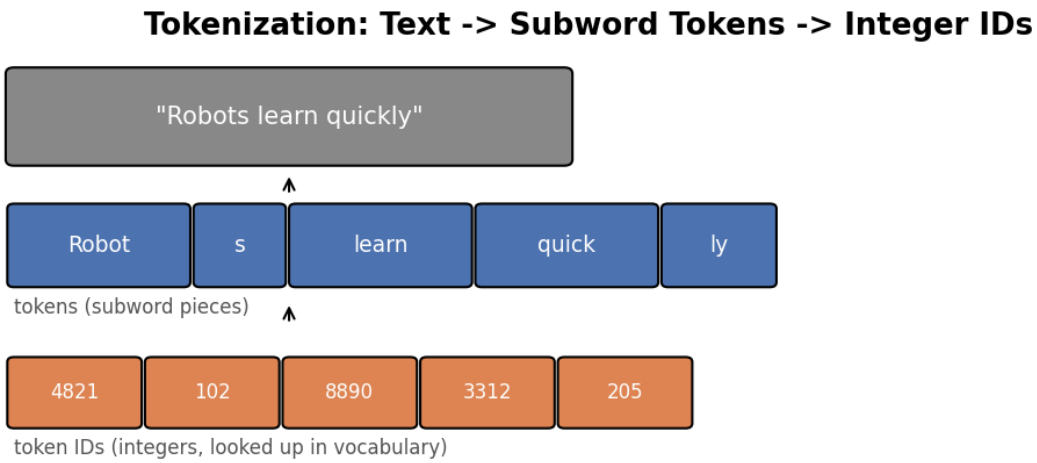


Figure 1: Tokenization: raw text is split into subword tokens, and each token is looked up in a vocabulary to produce a sequence of integer IDs — the actual input a Transformer receives.

2.2 Three Levels of Tokenization, and Why Subwords Won

Word-level: split on spaces/punctuation, one token per word. Simple, but the vocabulary must contain every word the model will ever see — new words (typos, rare names, made-up words) become an "unknown" token, permanently losing information. Vocabularies also balloon to hundreds of thousands of entries once you account for every inflection ("run," "runs," "running," "ran").

Character-level: one token per character. The vocabulary is tiny (dozens of symbols), and there's never an "unknown" token — anything can be spelled out character by character. The cost: sequences become very long (a 10-word sentence might become 50+ character tokens), and the model has to work much harder to reconstruct word-level and sentence-level meaning from individual letters.

Subword-level (the modern standard): a middle ground. Common whole words ("the," "robot") stay as single tokens; rare or complex words get split into meaningful pieces ("unhappiness" → "un" + "happi" + "ness"). This gets nearly all of word-level's efficiency (short sequences, meaningful chunks) while keeping character-level's flexibility (never truly "unknown" — an unfamiliar word can always be broken down into smaller known pieces, down to individual characters/bytes as a fallback).

2.3 How Subword Vocabularies Are Actually Built

The dominant algorithm is **Byte-Pair Encoding (BPE)**, and its logic is simple enough to trace by hand:

1. Start with every word split into individual characters.
2. Count every adjacent pair of symbols across the entire training corpus.
3. Merge the single most frequent pair into one new symbol.
4. Repeat steps 2–3 for a fixed number of merges (this number — e.g. 32,000 or 50,000 — becomes the vocabulary size).

Because merges are driven by frequency, extremely common sequences (like “ing” or “the”) get merged into single tokens quickly, while rare sequences stay split into smaller pieces. This is *exactly* why “robot**s**” split into “Robot” + “s” in the diagram above — “Robot” alone was frequent enough to earn its own token, while the plural “-s” ending, being productively attached to thousands of different words, is more efficiently kept as a separate, reusable piece.

WordPiece (used by BERT) and **SentencePiece** (used by many multilingual models) are close variants of the same core idea, differing mainly in the exact merge-selection criterion and whether they operate on raw bytes or Unicode characters.

2.4 Token IDs and the Vocabulary

Once the vocabulary is fixed (a list of, say, 50,000 possible tokens), every token is simply its **index** in that list — an integer. “Robot” might be ID 4821, “the” might be ID 15 (common tokens tend to get low IDs, though this isn’t a strict rule). The entire rest of the Transformer only ever sees this sequence of integers — the actual characters are gone by this point, replaced by lookups.

2.5 Special Tokens

Every vocabulary reserves a handful of IDs for tokens that aren’t literal text:

- `[CLS]` / `<s>`: often prepended to mark the start of a sequence, and in BERT-style encoders, its final representation is used as a summary of the whole input for classification (covered further in Chapter 11’s Vision Transformer discussion, which reuses this exact idea)
- `[SEP]` / `</s>`: marks the end of a sequence, or separates two sequences packed together (e.g. a question and a passage)
- `[PAD]`: a filler token used to make sequences in a batch the same length, and masked out of the loss/attention so it has no effect on real predictions
- `[UNK]`: the fallback for anything that genuinely can’t be represented (rare with modern subword/byte-level tokenizers, common with older word-level ones)
- `[MASK]`: used specifically during BERT-style masked-language-model training (Chapter 9) — the token temporarily hides a real token so the model can be trained to predict it

2.6 A Worked Example, Traced by Hand

Take the word “unhelpful” with a hypothetical trained vocabulary containing “un”, “help”, “ful”, “un helpful” (not present as a whole word). Tokenization would output: `["un", "help", "ful"]` → IDs `[812, 3390, 205]`. Note that this is a *lossless, reversible* process — concatenating “un” + “help” + “ful” reconstructs “unhelpful” exactly. This reversibility is a hard requirement: the model’s final output also has to be converted back from token IDs into readable text, and that only works if tokenization is exactly invertible.

2.7 Why This Chapter Matters More Than It Seems

Tokenization quietly shapes everything downstream: the *sequence length* the Transformer sees (a badly-fit tokenizer can make sequences 2–3x longer than necessary, directly increasing the $O(n^2)$ attention cost from Chapter 5), and even *what a model can and can’t do well* — a

model tokenizing numbers digit-by-digit versus in awkward multi-digit chunks will have measurably different arithmetic ability, a well-documented quirk in real LLMs traceable directly back to this first, easy-to-overlook step.

3 Chapter 3 — Embeddings: Giving Each Token a Meaning

3.1 From Integer to Vector

A token ID like 4821 is just an arbitrary index — it carries no information about what "Robot" *means*, or how it relates to "machine" or "android." The **embedding layer** fixes this: it's a lookup table, one learned vector per vocabulary entry, mapping every token ID to a dense vector of (typically) several hundred to a few thousand numbers.

$$\text{embedding}(4821) = [0.12, -0.87, 0.33, \dots, 0.05] \in \mathbb{R}^{d_{model}}$$

Mechanically, this is nothing more than a matrix $E \in \mathbb{R}^{V \times d_{model}}$ (where V is vocabulary size) and a **row lookup**: $\text{embedding}(id) = E[id, :]$. This matrix is a learned parameter, trained via backpropagation exactly like every other weight in the network — nobody hand-assigns these numbers.

3.2 The Analogy: A Map of Meaning

Think of the embedding space as a huge, high-dimensional map, where every possible token has a fixed location. Training pulls related concepts closer together and pushes unrelated ones apart, purely as a side effect of the model learning to predict/understand text well. The now-classic illustration: in a well-trained embedding space, the vector arithmetic

$$\text{embedding}(\text{king}) - \text{embedding}(\text{man}) + \text{embedding}(\text{woman}) \approx \text{embedding}(\text{queen})$$

actually holds up reasonably well in practice — the *direction* from "man" to "woman" in this learned space captures something like "gender," and applying that same direction to "king" lands near "queen." Nobody programmed this relationship; it emerged purely from the statistics of how these words are used in enormous amounts of text.

3.3 Embedding Dimensionality

d_{model} — the length of each embedding vector — is a core hyperparameter of the whole Transformer, not just the embedding layer: every subsequent layer (attention, feed-forward) is built to operate on vectors of this exact size, and the same d_{model} carries through the entire depth of the network. Typical values: 512–768 for smaller models (BERT-base uses 768), scaling up to many thousands for large modern LLMs. Larger d_{model} means each token can encode richer, more nuanced information, at the direct cost of more parameters and more compute per layer.

3.4 Weight Tying: Reusing the Embedding Matrix

Many Transformers reuse the *same* embedding matrix E for two purposes: converting input token IDs into vectors at the start of the network, **and** converting the final hidden state back into a probability distribution over the vocabulary at the output (via $E^\top$, essentially running the lookup in reverse). This is called **weight tying**. It roughly halves the parameter count spent on vocabulary-sized matrices, and empirically tends to improve training — the intuition being that "how similar is this output vector to the word 'robot' " and "what does the word 'robot' mean as an input" are, at some level, the same underlying question.

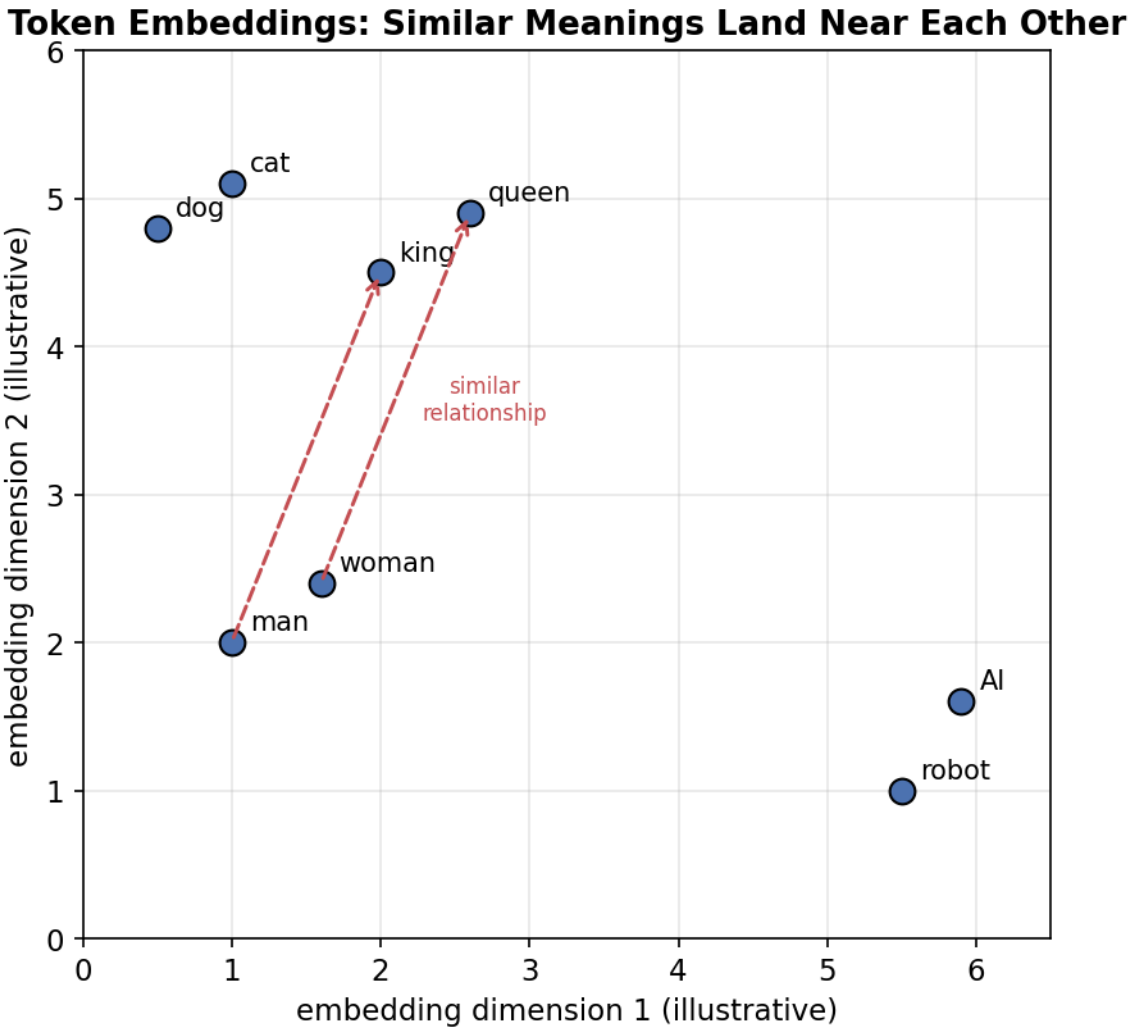


Figure 2: Token embeddings place similar meanings near each other in a high-dimensional space (shown here compressed to 2D for visualization). The relationship vector between related pairs (king→queen, man→woman) points in a similar direction — a well-known emergent property of trained embeddings.

3.5 A Note on Vision and Other Modalities

The same idea — map a discrete or raw input into a learned vector of size d_{model} before anything else happens — generalizes far beyond text tokens. Chapter 11 shows exactly this: a Vision Transformer doesn't look up a word in a vocabulary, but it does linearly project each image patch into a d_{model}-sized vector, functionally serving the exact same role as this chapter's embedding lookup. The core lesson to take from this chapter — *before a Transformer's attention layers can do anything, every input element must first become a fixed-size vector in a shared space* — is one of the most portable ideas in this entire book, resurfacing for images (Ch. 11), for combined vision+language (Ch. 12), and for robot actions (Ch. 15).

4 Chapter 4 — Positional Encoding: Teaching the Model About Order

4.1 The Problem This Solves

Here's a fact worth sitting with: **self-attention (Chapter 5), by itself, has no idea what order the tokens came in.** If you scrambled the words of a sentence and fed them into raw self-attention, it would compute the exact same set of pairwise relationships regardless of which token came first, second, or last — attention is a function of *content*, not *position*. But obviously "the dog bit the man" and "the man bit the dog" mean very different things, and a Transformer needs to tell them apart. Positional encoding is the fix: inject explicit position information into each token's representation before attention ever runs.

4.2 Sinusoidal Positional Encoding (the Original Approach)

The original Transformer paper adds a fixed (non-learned) vector to every token embedding, where each dimension of the vector is a sine or cosine wave at a different frequency:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right), \qquad PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

where pos is the token's position in the sequence and i indexes the embedding dimension.

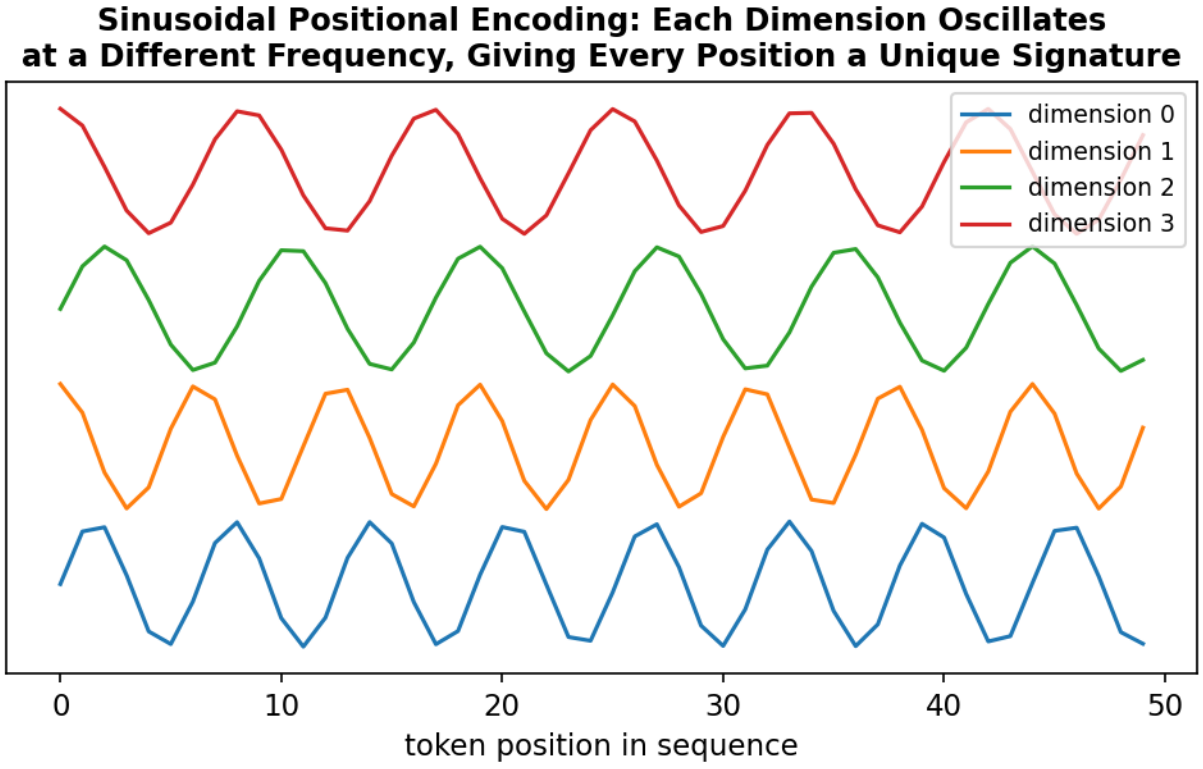


Figure 3: Sinusoidal positional encoding: each dimension oscillates at a different frequency across position. Low dimensions oscillate quickly (capturing fine-grained local position); high dimensions oscillate slowly (capturing coarse, long-range position). The combination across all dimensions gives every position a unique "fingerprint."

Why sine and cosine waves, specifically, and not just the raw position number? Two deliberate reasons:

1. **Bounded values.** Position 50,000's raw index is a huge number that would dominate and destabilize the embedding; sin/cos always stay in $[-1, 1]$, regardless of how long the sequence gets.
2. **Relative position becomes a simple linear transformation.** Because of a trigonometric identity, PE_{pos+k} can be expressed as a fixed linear function of PE_{pos} (for any fixed offset k) — meaning the network has an easy mathematical handle on "how far apart are these two tokens," not just "what are their absolute positions." This was believed to help the model generalize to relative relationships regardless of absolute position.

4.3 Learned Positional Embeddings

A simpler alternative: just make positional encoding another lookup table, exactly like the token embedding table from Chapter 3, but indexed by position instead of token identity — a fully learned vector for position 0, another for position 1, and so on. BERT uses this approach. It's simpler and empirically works about as well for a fixed maximum sequence length, but has an obvious weakness: it has literally never seen (and has no learned vector for) any position beyond whatever maximum length was used during training, so it cannot generalize to longer sequences at all.

4.4 Modern Approaches: RoPE and ALiBi

Most current large language models use neither of the above directly, favoring approaches that generalize better to sequence lengths not seen during training:

- **RoPE (Rotary Position Embedding)**: instead of *adding* a positional vector, RoPE *rotates* the query and key vectors (from Chapter 5) by an angle proportional to their position, before computing attention scores. The elegant consequence: the dot product between a rotated query and a rotated key ends up depending only on their *relative* position (the angle difference), not their absolute positions — relative position falls out of the mechanism directly, rather than being an emergent property the model has to learn. RoPE is used in most modern open-weight LLMs (the LLaMA family and similar).
- **ALiBi (Attention with Linear Biases)**: skips modifying the embeddings entirely, and instead adds a fixed penalty directly to the raw attention scores, proportional to the distance between the query and key positions — closer tokens get less penalty, distant tokens get more. Extremely simple, and notably good at **extrapolating** to sequences much longer than anything seen during training, since the penalty formula has no learned parameters that could "run out" past a certain length.

4.5 Why This Chapter Is Short But Not Small

Positional encoding is a small piece of code relative to attention or the feed-forward layers, but the *choice* of scheme has an outsized, very practical effect: it's the single biggest factor in whether a trained model can handle sequences longer than it was trained on (a property called **length extrapolation**) — a genuinely important, actively researched property for any application (long documents, long robot trajectories, long conversations) where the deployed sequence length can't be perfectly predicted ahead of time.

5 Chapter 5 — Self-Attention: The Heart of the Transformer

5.1 The Intuition First, Before Any Math

Take the sentence "The robot moved because it was heavy." When the model processes the word "it," it needs to figure out what "it" refers to — "the robot" or something implied by "heavy"? Self-attention lets the token "it" directly compare itself against every other token in the sentence and compute a **relevance score** for each — then blend information from the other tokens weighted by those scores.

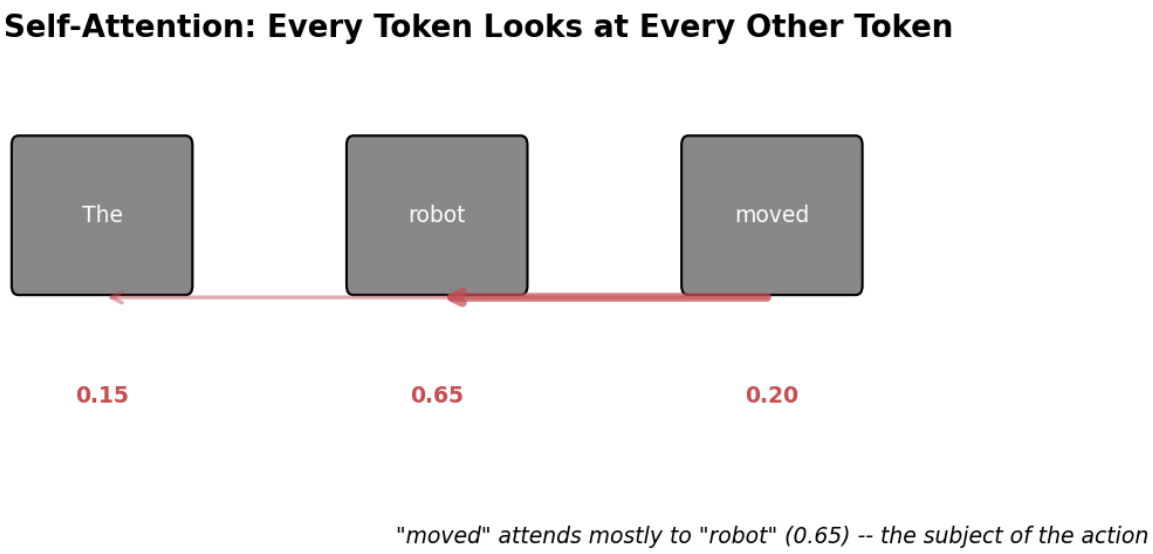


Figure 4: Self-attention: the token "moved" compares itself against every token in the sentence, computing a weight for each. Here it attends most strongly (0.65) to "robot" — correctly identifying the subject performing the action.

This is exactly the mechanism you use, wordlessly, every time you resolve a pronoun or figure out which adjective modifies which noun in a long sentence. Self-attention formalizes "glance back at whatever's relevant" into something differentiable and trainable.

5.2 Query, Key, Value — A Second Analogy: A Library Search

Every token produces three different vectors from its embedding, via three separate learned weight matrices:

- **Query (Q)**: "what am I looking for?" — think of this as the search terms you type into a library catalog
- **Key (K)**: "what do I contain?" — think of this as the subject tags/index terms attached to every book in the library
- **Value (V)**: "what do I actually hand over if I'm picked?" — think of this as the actual content of the book itself, handed to you if your search matched its tags

A token's **query** is compared against every other token's **key** to compute a match score (like

matching your search terms against every book's tags); those scores become weights; and the final output is a weighted blend of every token's **value** (like combining the actual content of the best-matching books, weighted by how well each one matched).

5.3 The Formula, Explained Term by Term

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V$$

X is the matrix of all token embeddings (plus positional encoding) stacked into rows — one row per token. W_Q, W_K, W_V are learned weight matrices, **the actual thing the network trains** in this layer — everything else in this chapter is a fixed computation once these three matrices are set.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

Breaking this into its exact steps, matching the diagram:

1. $QK^\top$: for every pair of tokens, compute the dot product between one token's query and another's key. A large dot product means "these vectors point in a similar direction" — i.e., a strong match. This produces an $n \times n$ matrix of raw relevance scores, where n is the sequence length.
2. **Divide by** $\sqrt{d_k}$: as the key dimension d_k grows, dot products naturally grow larger in magnitude too (summing more terms), which would push softmax into a regime with extremely small gradients (nearly one-hot outputs, hard to learn from). Dividing by $\sqrt{d_k}$ keeps the scores in a numerically well-behaved range regardless of dimensionality — a purely practical fix, not a conceptual one.
3. **softmax**: converts the raw scores for each token's row into a proper probability distribution — all weights positive, summing to 1 across all the tokens being attended to. This is what turns "relevance scores" into "how much of each token's value to actually use."
4. **Multiply by** V: take the weighted combination of every token's value vector, using the softmax weights just computed. This is the layer's actual output for that token — a blend of information pulled from wherever the query decided was relevant.

5.4 A Fully Worked Numeric Example

To make this completely concrete, work through a tiny example with just 2 tokens and 2-dimensional vectors (real models use hundreds of dimensions and many tokens, but the arithmetic is identical in kind).

Suppose after projection:

$$Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad V = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$$

Step 1 — $QK^\top$**:**

$$QK^\top = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Step 2 — scale by $\sqrt{d_k} = \sqrt{2} \approx 1.41$**:**

$$\begin{bmatrix} 0.71 & 0 \\ 0 & 0.71 \end{bmatrix}$$

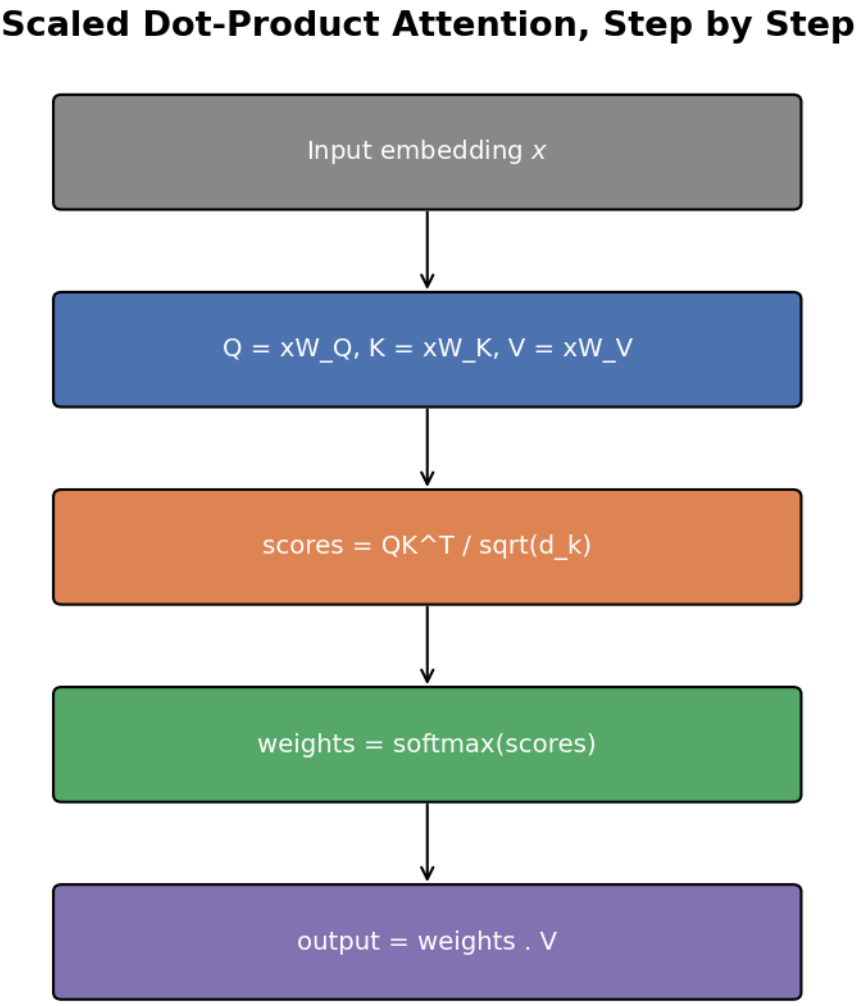


Figure 5: Scaled dot-product attention, traced step by step: project into Q/K/V, compute similarity scores, scale, normalize into weights via softmax, then blend the values.

Step 3 — softmax each row: row 1 is $[0.71, 0]$ → softmax gives approximately $[0.67, 0.33]$ (the larger score gets more weight, but softmax never assigns exactly 0 or 1 — some weight always leaks to the other option). By symmetry, row 2 gives $[0.33, 0.67]$.

Step 4 — multiply by V: token 1's output = $0.67 \times [2, 0] + 0.33 \times [0, 3] = [1.34, 0.99]$. Token 2's output, by the same symmetric logic, comes out to $[0.66, 2.01]$.

Notice the result: token 1 ends up mostly reflecting token 1's own value $[2, 0]$ (weight 0.67) with a smaller contribution from token 2's value (weight 0.33) — exactly the "mostly yourself, some relevant context from elsewhere" blending that makes self-attention useful. In a real trained model, the weight matrices W_Q, W_K, W_V are shaped by training so that these weights land on genuinely meaningful relationships (subject-verb agreement, pronoun resolution, and so on), rather than the arbitrary identity-like matrices used here purely to keep the arithmetic traceable by hand.

5.5 Why "Self" Attention, Specifically

This chapter's mechanism is called **self**-attention because Q, K, and V are all derived from the *same* input sequence X — every token attends to (potentially) every other token in the same sequence it belongs to. Chapter 8 introduces **cross-attention**, the same mechanism with one change: queries come from one sequence, while keys/values come from a *different* sequence — the mechanism that lets a decoder look back at an encoder's output.

6 Chapter 6 — Multi-Head Attention: Several Perspectives at Once

6.1 The Limitation of a Single Attention Computation

Chapter 5's self-attention computes exactly *one* set of relevance scores per pair of tokens. But language (and, as later chapters show, images and robot state) has many *simultaneous, different kinds* of relationships worth tracking — grammatical relationships, coreference (which pronoun refers to which noun), topical similarity, positional adjacency. Forcing all of that through a single attention computation means averaging together relationships that might be better kept separate.

6.2 The Fix: Run Attention Multiple Times, in Parallel, in Smaller Subspaces

Instead of one large attention computation, multi-head attention splits Q, K, V into h smaller pieces (called **heads**), runs Chapter 5's full scaled dot-product attention independently within each piece, and then recombines the results:

$$\text{head}_i = \text{Attention}(XW_Q^{(i)}, XW_K^{(i)}, XW_V^{(i)})$$

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)\, W_O$$

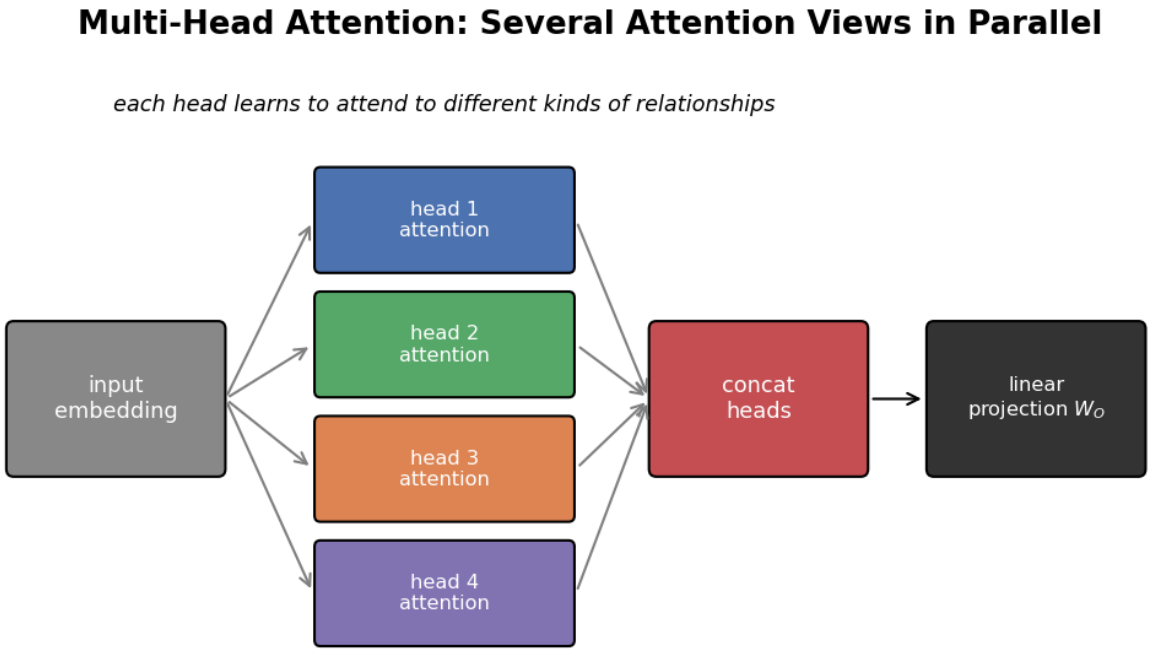


Figure 6: Multi-head attention: the input is projected into several smaller subspaces (heads), each running its own independent attention computation, then the results are concatenated and linearly projected back to the model's full dimension.

Each head has its **own separate** $W_Q^{(i)}, W_K^{(i)}, W_V^{(i)}$ — meaning each head is free to learn to specialize in a different kind of relationship, entirely through the ordinary pressure of gradient

descent minimizing the training loss (nobody assigns heads their specialization by hand). W_O is one more learned matrix that projects the concatenated output of all heads back down to the model's standard dimension d_{model}, so the output of this whole layer is the same shape as its input — a requirement for stacking many of these blocks (Chapter 7).

6.3 What Different Heads Actually Learn — Documented, Interpretable Patterns

This isn't just a theoretical claim — analyzing trained Transformers' attention weights has repeatedly found interpretable specialization across heads:

- Some heads consistently attend to the **immediately preceding or following token** — capturing local, positional structure
- Some heads specialize in **syntactic relationships** — e.g. a verb attending strongly to its direct object, regardless of how many words separate them
- Some heads track **coreference** — a pronoun attending back to the noun it refers to, sometimes across a considerable distance
- Some heads (especially in later layers) capture more diffuse, semantic/topical similarity rather than any crisp grammatical relationship

No individual head is told to do any of this — it's an emergent consequence of many heads competing, during training, to each carve out some useful, non-redundant slice of the attention computation's total capacity.

6.4 Practical Sizing

The head dimension is simply d_{model}/h — e.g. a model with $d_{model} = 768$ and $h = 12$ heads gives each head a working dimension of 64. Splitting into more, smaller heads doesn't increase the total parameter count of the projections (the total width across all heads combined still adds up to d_{model}) — it's a genuine architectural trade-off between fewer, higher-capacity heads and more, narrower, more specialized heads, and $h = 8$ to $h = 16$-ish is a common practical range, though large modern models scale this up considerably alongside everything else.

The Architectural Trade-OffChoosing the number of heads is a balance between two styles of thinking:Fewer, larger heads: Each head has a higher capacity. They can understand complex, deep relationships, but they can only focus on a few things at once.

More, smaller heads: Each head is narrower and more specialized. They can track many different things at the same time (like grammar, tone, and facts), but each head has less individual power.

7 Chapter 7 — The Full Transformer Block

7.1 Assembling the Pieces

Chapters 2–6 covered every ingredient. This chapter assembles them into the actual repeating unit that gets stacked, many times over, to form a real Transformer.

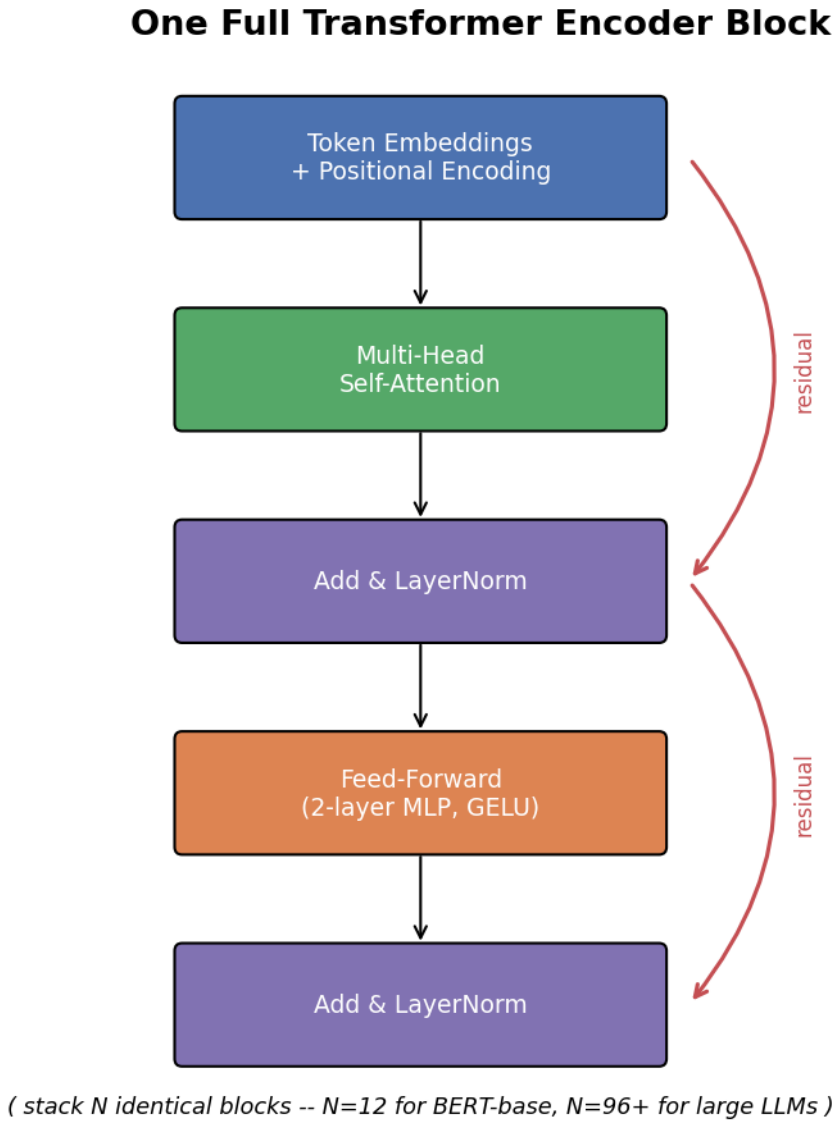


Figure 7: One complete Transformer encoder block: multi-head self-attention and a feed-forward sublayer, each wrapped in a residual connection and followed by Layer Normalization. Real models stack many of these blocks — 12 for BERT-base, 96+ for large modern LLMs.

7.2 The Two Sublayers

Every block has exactly two sublayers, each wrapped identically:

$$x' = \text{LayerNorm}\big(x + \text{MultiHeadAttention}(x)\big)$$

$$x'' = \text{LayerNorm}\big(x' + \text{FeedForward}(x')\big)$$

Sublayer 1 — Multi-Head Self-Attention (Chapter 6): lets every token gather information from every other token, mixing information *across positions*.

Sublayer 2 — Feed-Forward Network: a plain 2-layer MLP, applied **independently and identically to every token position** (no mixing across positions happens here — that already happened in sublayer 1):

$$\text{FFN}(x) = W_2\,\text{GELU}(W_1 x + b_1) + b_2$$

The inner dimension is typically **4× wider** than d_{model} (e.g. $d_{model} = 768$ expands to 3072 inside the FFN, then projects back down to 768). This sublayer, despite being architecturally simpler than attention, actually holds the majority of a Transformer's raw parameter count and per-token compute — attention decides *what to combine*, while the FFN does much of the actual "thinking" *once information has been combined*.

7.3 Why the Residual Connections Are Not Optional

Every sublayer is wrapped as $x + \text{Sublayer}(x)$, not just $\text{Sublayer}(x)$ — the exact same residual/skip-connection idea from ResNet (covered in the Deep Learning textbook, Chapter 4.5), applied here across *Transformer* depth instead of *CNN* depth. Stacking dozens or hundreds of these blocks without residual connections would make training practically impossible — gradients would have to pass through that many stacked nonlinear transformations, and (echoing the exact vanishing-gradient logic from the RNN book's Chapter 3, just applied across network depth here instead of across time) the signal reaching early layers would shrink toward uselessness. The residual connection gives gradients a direct, mostly-unimpeded path all the way back to the input, which is what makes 96+-layer Transformers trainable at all.

7.4 Why Layer Normalization, Specifically (Not Batch Normalization)

LayerNorm normalizes each individual token's vector independently (across its own d_{model} features), rather than across a batch of examples the way BatchNorm does (Deep Learning textbook, Chapter 3.4). This matters for two concrete reasons specific to sequence models: (1) sentences/sequences in a batch have different lengths, and batch statistics computed across padding tokens would be meaningless or actively harmful; (2) at inference time, sequences are frequently processed one at a time (e.g. generating one token at a time during autoregressive decoding, Chapter 8), where there's no "batch" to compute statistics over at all. LayerNorm sidesteps both issues entirely by never depending on other examples in the batch.

Pre-Norm vs. Post-Norm: the equations above show *post-norm* (normalize after adding the residual) — this is what the original 2017 paper used. Most modern large-scale Transformers instead use **pre-norm** (normalize *before* each sublayer, i.e. $x + \text{Sublayer}(\text{LayerNorm}(x))$), which has been found empirically to make training noticeably more stable at large depth, at a very minor cost to final performance in some settings — a widely adopted practical adjustment, though the core computation is otherwise identical.

LayerNorm (Layer Normalization) standardizes data within a single sentence, while BatchNorm (Batch Normalization) mixes data across different sentences.

Pre-Norm makes training incredibly stable: By cleaning the data before it enters the next layer, the model maintains a smooth, safe pathway for information to flow. This allows engineers to build massive, super-deep AI models without them breaking.

7.5 Stacking Blocks

A full Transformer is simply N of these blocks stacked directly on top of each other, each one's output feeding the next block's input unchanged in shape (which is exactly why every sublayer is carefully designed to output something the same shape as d_{model} — it has to, or stacking wouldn't be possible). Deeper stacks (more blocks) generally give the model more capacity to build increasingly abstract representations layer by layer — echoing the same "depth builds hierarchy" principle from CNNs (Deep Learning textbook, Chapter 4.7), just realized here through attention and feed-forward sublayers instead of convolutions.

8 Chapter 8 — Three Flavors: Encoder-Only, Decoder-Only, Encoder-Decoder

8.1 Why the Same Building Block Splits Into Three Architectures

Chapter 7's block is a general-purpose unit. What differs across the three major Transformer families is **which tokens are allowed to attend to which other tokens**, and **whether there are one or two separate stacks of blocks**. This single design choice — the attention *mask* — is responsible for the entire split between "understanding" models and "generating" models.

8.2 Encoder-Only (BERT-Style): Full, Bidirectional Attention

Every token can attend to every other token in the sequence, in both directions — past and future, all at once. This requires the **entire input to be available upfront**, which is fine for tasks like classification or tagging, where the whole sentence already exists before the model needs to produce an answer.

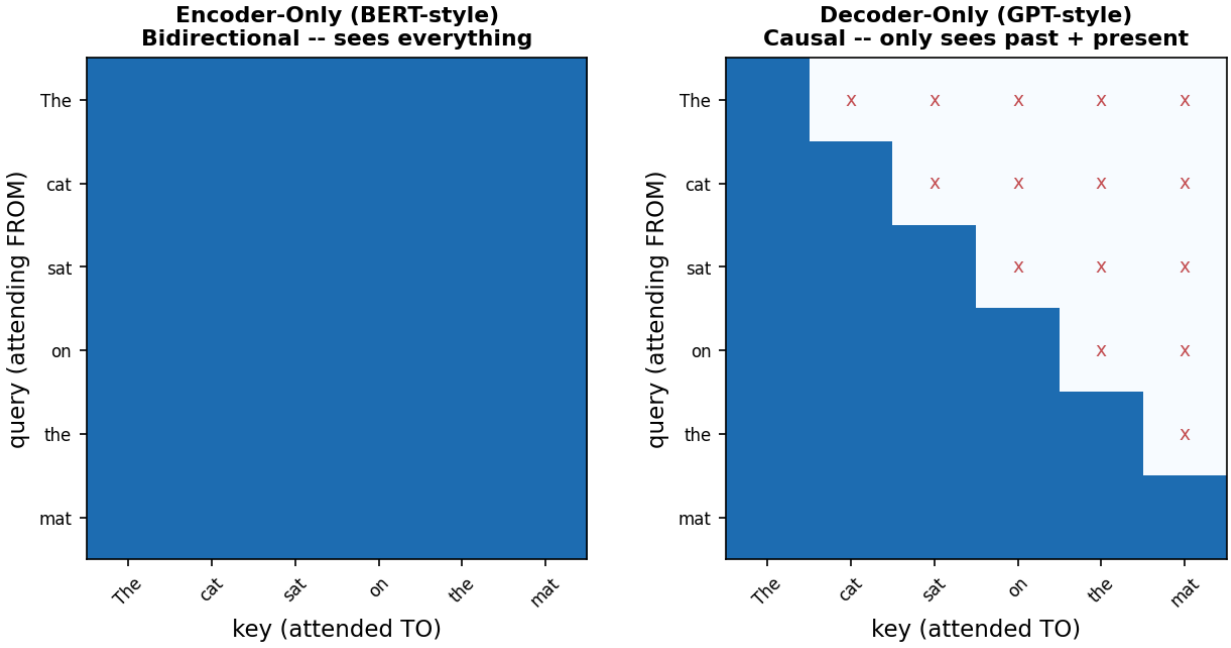


Figure 8: Left: encoder-only attention is fully bidirectional — every token (row) attends to every other token (column), shown as fully shaded. Right: decoder-only attention is causal — a token can only attend to itself and earlier tokens; the upper-right triangle (future tokens) is masked out entirely (marked with x).

Training objective — Masked Language Modeling (MLM): randomly hide (replace with `[MASK]`, Chapter 2) about 15% of input tokens, and train the model to predict the original token at each masked position, using context from **both directions**. This bidirectional context is exactly what makes encoder-only models strong at *understanding* tasks (classification, named entity recognition, producing high-quality sentence embeddings) — but structurally unsuitable for open-ended generation, since generation requires producing tokens one at a time without knowing the future ones yet.

1. Encoder-Only Models (The "Readers")Encoder-only models are designed to read and understand the context of an entire sentence at once. They look at all the words in a text simultaneously (bi-directional processing) to get a deep, complete picture of what the text means.

2. Decoder-Only Models (The "Writers")Decoder-only models are designed to predict and generate the next word in a sequence. Unlike the encoder, a decoder reads text from left to right and is masked, meaning it cannot look ahead to see future words—it only knows what has already been written.

8.3 Decoder-Only (GPT-Style): Causal (Masked) Attention

Each token can only attend to **itself and earlier tokens** — never anything later in the sequence. This is enforced by literally setting the attention score to $-\infty$ (before softmax) for every position pair where the key comes after the query, guaranteeing a zero attention weight for any "future" token once softmax is applied.

Training objective — Next-Token Prediction: given tokens $1, \ldots, t$, predict token $t+1$; repeat this prediction target at every position in the sequence simultaneously (using the causal mask to make sure position t's prediction genuinely only used information from positions $\leq t$). This is the architecture behind essentially every modern large language model and chat assistant — it's a natural fit because the training objective *is* the generation process: at inference time, the model simply keeps calling itself on its own growing output, one new token at a time (this is called **autoregressive generation**).

8.4 Encoder-Decoder (T5-Style, and the Original 2017 Architecture)

Two separate stacks of blocks: an encoder (bidirectional, as above) processes the full input; a decoder (causal, as above) generates the output autoregressively, with one addition — each decoder block also includes a **cross-attention** sublayer, where the decoder's tokens act as queries, but the *keys and values come from the encoder's final output*.

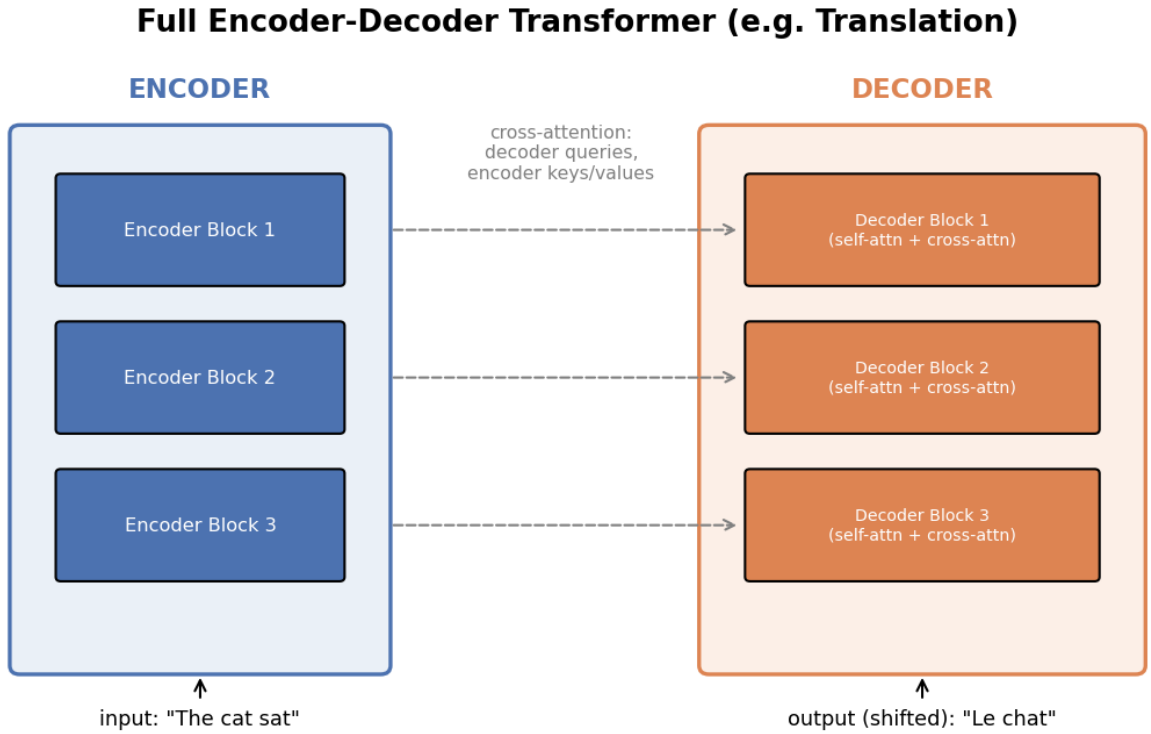


Figure 9: The full encoder-decoder Transformer. The encoder builds a bidirectional representation of the input; each decoder block both self-attends (causally, over its own generated output so far) and cross-attends (over the encoder's full output) before producing the next token.

This is the direct, natural successor to the RNN-based sequence-to-sequence architecture from the RNN book (Chapter 8) — the encoder still compresses an input, the decoder still

generates an output conditioned on it — except the RNN book's single fixed-size context vector bottleneck is gone entirely: the decoder can cross-attend to *every* encoder position, not just one compressed summary. This is the exact fix the RNN book's Chapter 8 predicted attention would eventually deliver, now fully realized without any recurrence left at all.

8.5 Choosing Among the Three, in Practice

Architecture	Attention pattern	Best suited for	Examples
Encoder-only	Bidirectional	Classification, embeddings, understanding	BERT, RoBERTa
Decoder-only	Causal	Open-ended generation, chat, code	GPT-family, LLaMA, most modern LLMs
Encoder-decoder	Bidirectional (encoder) + causal + cross-attn (decoder)	Translation, summarization — explicit input→output transduction	T5, the original Transformer, many earlier machine translation systems

A notable industry trend: decoder-only models have become the dominant default for most large-scale applications in recent years — even tasks that seem naturally encoder-decoder shaped (like summarization) are now frequently handled by simply prompting a decoder-only model with the input followed by an instruction, rather than building a dedicated two-stack architecture. Encoder-only and true encoder-decoder architectures remain important, especially where fast, non-generative embeddings are the actual goal (retrieval, search, RAG pipelines).

9 Chapter 9 — How Transformers Are Actually Trained

9.1 Pretraining vs. Fine-Tuning: The Two-Stage Recipe

Almost every practically deployed Transformer today goes through two distinct stages:

Pretraining: train on a huge amount of raw, unlabeled text (or images, or other modality) using a self-supervised objective that requires no human annotation — masked language modeling for encoders (Chapter 8), next-token prediction for decoders. This stage is where the vast majority of compute and data is spent, and it's what gives the model its broad, general capability.

Fine-tuning: continue training (usually for far less time, on far less data) on a specific labeled task or a curated instruction dataset, adapting the general capability from pretraining toward a specific use case. This is dramatically more data-efficient than training a task-specific model from scratch, since the model already "knows" a great deal about language (or images) before fine-tuning even starts — precisely the same self-supervised-pretraining-then-fine-tune logic covered in the Deep Learning textbook, Chapter 8.5, and in the RL+IL book's discussion of pretraining before RL fine-tuning.

9.2 Instruction Tuning and Alignment (Brief, for Context)

For chat-style decoder-only models specifically, there's commonly a further stage: **instruction tuning** (supervised fine-tuning on instruction→response pairs, teaching the model to follow instructions rather than just continue text in a generic way), often followed by a preference-alignment stage like **RLHF** or **DPO** (both covered in depth in the RL+IL book, Chapter 11) — steering the model's *style and behavior*, not just its raw next-token prediction competence.

9.3 Teacher Forcing (Inherited Directly From the RNN World)

When training a decoder-only or encoder-decoder model to generate a sequence, the exact same **teacher forcing** technique from the RNN book (Chapter 3) applies: feed the *ground-truth* previous token as input at every position during training, rather than the model's own (possibly wrong) prediction — this lets every position's loss be computed in parallel in a single forward pass (since the causal mask from Chapter 8 already ensures position t only sees positions $\leq t$), rather than requiring the slow, sequential, one-step-at-a-time generation that would otherwise be needed. This parallelizability during training — even though generation at inference time is still sequential — is one of the underappreciated practical reasons Transformers train so much faster than RNNs at the same effective sequence length.

9.4 The Loss Function

For next-token prediction: standard cross-entropy loss (Deep Learning textbook, Chapter 2.2) between the predicted probability distribution over the vocabulary at each position, and the actual next token, averaged over every position in every sequence in the batch.

9.5 Training at the Scale Modern Transformers Require

Everything in the Deep Learning textbook's Chapter 8 applies here directly and is, in practice, largely *written with Transformers specifically in mind*: mixed precision training, gradient checkpointing, and distributed training (data/model/pipeline parallelism, FSDP/ZeRO) are essentially mandatory engineering requirements once a Transformer's parameter count reaches the billions — no single GPU can hold the full set of weights, gradients, and optimizer

Cross-entropy loss measures how well a model predicts the exact next token by penalizing it when it assigns a low probability to the correct word.

The loss is calculated at every single word position across all sentences in a batch, then averaged together to create one final number used to update the model's weights .

states for a large model, let alone train it in a reasonable amount of time. **FlashAttention** (Chapter 13 of this book) is a Transformer-specific efficiency technique important enough to warrant its own dedicated discussion later.

9.6 A Note on Data Quality and Scale

Empirically, Transformer performance has been found to scale remarkably predictably with three quantities: model size (parameter count), dataset size, and compute budget — a relationship formalized in what are commonly called **scaling laws**. This predictability is part of why large labs can plan multi-million-dollar training runs with reasonable confidence in the resulting model's performance ahead of time, and it's a genuinely distinctive property of this architecture family relative to most prior deep learning approaches, where scaling behavior was considerably less predictable.

10 Chapter 10 — A Survey of Transformer Types

This chapter is a map of the landscape — brief entries on the major named Transformer variants, so you recognize them and know roughly where they fit relative to the architecture covered in Chapters 5–9. Several are explored in much greater depth in later chapters (marked below).

10.1 Text-Only Transformers

- **BERT** (Bidirectional Encoder Representations from Transformers): encoder-only, masked language modeling, the model that established encoder-only pretraining as a powerful general strategy for understanding tasks
- **GPT family**: decoder-only, next-token prediction, the lineage behind most modern chat-oriented LLMs
- **T5** (Text-to-Text Transfer Transformer): encoder-decoder, reframes *every* NLP task (classification, translation, summarization) as a text-to-text problem, sharing one architecture and objective across all of them
- **RoBERTa**: BERT with a more carefully tuned training recipe (more data, longer training, some objective simplifications) — a useful reminder that training recipe, not just architecture, matters enormously
- **ELECTRA**: replaces BERT's "predict the masked token" objective with "detect which tokens were replaced by a small generator network" — a more sample-efficient pretraining signal, since every token position provides a training signal rather than only the ~15% that were masked

10.2 Vision Transformers (Full Deep Dive in Chapter 11)

- **ViT** (Vision Transformer): the original — split an image into patches, treat them as a token sequence, feed into a standard Transformer encoder
- **DeiT** (Data-efficient image Transformer): shows ViT-style models can be trained effectively on much smaller datasets than the original ViT paper required, via better training recipes and a distillation token
- **Swin Transformer**: computes attention only within local windows (not globally across the whole image), then shifts the windows between layers — dramatically more compute-efficient for high-resolution images and dense prediction tasks (detection, segmentation) than full global attention
- **BEiT / MAE (Masked Autoencoders)**: bring BERT-style masked pretraining to vision — mask out image patches, train the model to reconstruct them, entirely self-supervised
- **DINO / DINOv2**: self-distillation-based self-supervised training, producing general-purpose visual features (without any labels at all) that transfer remarkably well across many downstream vision tasks

10.3 Multimodal Transformers (Full Discussion in Chapter 12)

- **CLIP**: a vision encoder and a text encoder trained jointly via contrastive learning to place matching image-text pairs near each other in a shared embedding space — the foundation many later multimodal systems build on
- **Flamingo, BLIP, LLaVA**: Vision-Language Models (VLMs) that connect a vision encoder's output into a large language model's input space, enabling image-conditioned text generation (captioning, visual question answering, general "chat about this image")

10.4 Robotics-Oriented Transformers (Full Discussion in Chapter 15)

- **RT-1, RT-2**: Vision-Language-Action models — extend a VLM so it outputs robot actions (as tokens) instead of only text
- **OpenVLA**: an open-weight Vision-Language-Action model trained on large, diverse, cross-embodiment robot demonstration datasets
- **Decision Transformer**: reframes reinforcement learning itself as a sequence modeling problem — given a sequence of past states, actions, and *desired future return*, predict the next action, using a standard causal decoder-only Transformer with no explicit value function or policy gradient at all

10.5 Efficient / Long-Context Transformers (Full Discussion in Chapter 13)

- **Longformer, BigBird**: sparse/windowed attention patterns designed specifically to process much longer sequences than standard full attention can afford
- **Reformer, Linformer, Performer**: various approaches to approximate or restructure the attention computation to scale closer to linearly rather than quadratically with sequence length
- **Mixture of Experts (MoE) Transformers** (e.g. Mixtral, and the expert-routing layers inside several recent large frontier models): replace the dense feed-forward sublayer (Chapter 7) with many parallel "expert" sub-networks and a router that activates only a small subset per token — decoupling total parameter count from per-token compute cost

10.6 The Common Thread

Every single one of the models above is built from the exact same core components covered in Chapters 2–7 of this book — tokenization (or an input-specific equivalent), embeddings, positional encoding, multi-head self-attention, and a feed-forward sublayer, wrapped in residuals and LayerNorm. What varies is: which modality the tokens come from, whether attention is bidirectional or causal, whether there's one stack or two, and — for the efficiency-oriented variants — exactly which pairs of positions are allowed to attend to each other at all. Understanding Chapters 2–9 deeply means you can read the name of essentially any Transformer variant released and immediately know what to ask about it: *what's the input modality, what's the attention pattern, and what's the training objective?*

11 Chapter 11 — Vision Transformer (ViT): Deep Dive

11.1 The Provocative Original Claim

The Vision Transformer paper's title makes its argument in five words: *"An Image is Worth 16x16 Words."* The claim: you don't need convolutions (the architecture that had dominated computer vision for a decade — see the Computer Vision material in earlier chapters of this series) at all. Treat an image exactly like a sentence — as a sequence of tokens — and feed it into the *exact same* Transformer encoder architecture from Chapter 7, completely unmodified.

11.2 Step 1 — Splitting the Image Into Patches

An input image (say, 224×224 pixels) is cut into a grid of fixed-size, non-overlapping patches — commonly 16×16 pixels each, giving $(224/16)^2 = 196$ patches total. Each patch is flattened from a small 2D grid of pixels into a single 1D vector.

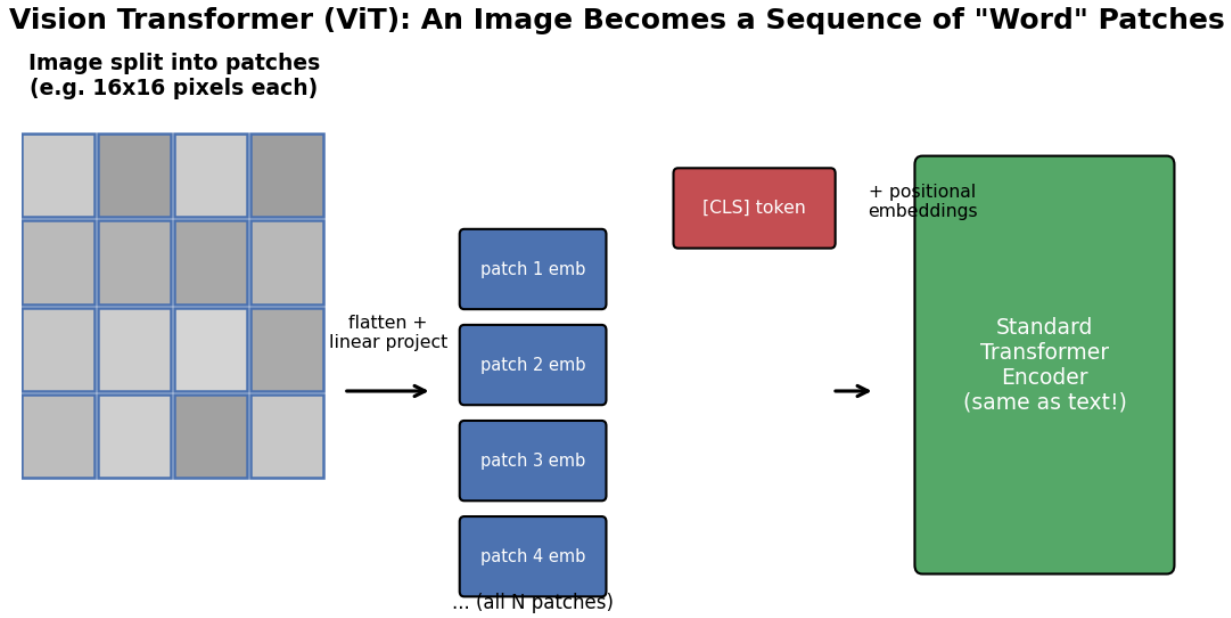


Figure 10: An image is split into fixed-size patches, each patch is flattened and linearly projected into an embedding vector — playing exactly the same structural role as a word's token embedding in Chapter 3 — a learnable [CLS] token is prepended, positional embeddings are added, and the whole sequence is fed into a completely standard Transformer encoder.

11.3 Step 2 — Linear Projection: The "Tokenization" of Vision

Each flattened patch (a vector of $16 \times 16 \times 3 = 768$ raw pixel values, for an RGB image) is passed through a single learned linear layer, projecting it into a d_{model}-sized embedding — **structurally identical to Chapter 3's token embedding lookup**, except a continuous linear projection replaces the discrete lookup table, since a patch is continuous pixel data rather than a discrete vocabulary entry. This is the entire "tokenization" step for vision: no analogue of Chapter 2's subword vocabulary is needed at all, since patches are naturally continuous and don't need to be mapped onto a fixed discrete symbol set first.

11.4 Step 3 — The [CLS] Token, Borrowed Directly From BERT

A single extra learnable embedding vector — the `[CLS]` token from Chapter 2 — is prepended to the sequence of patch embeddings. It doesn't correspond to any actual image patch; it exists purely to serve as an aggregation point. Through the self-attention layers, this token can attend to (and gather information from) every real patch in the image, and by the final layer, its representation is treated as a summary of the whole image.

11.5 Step 4 — Positional Embeddings

Because self-attention has no inherent sense of spatial arrangement (exactly as Chapter 4 explained for text), a learned positional embedding is added to each patch embedding, encoding *where in the grid* that patch came from. Without this step, ViT would have no way to distinguish "sky at the top of the image" from "sky randomly placed at the bottom" — the set of patches would be identical, only their arrangement differs.

11.6 Step 5 — A Completely Standard Transformer Encoder

From here, the sequence of (patch + position) embeddings, with the `[CLS]` token prepended, is fed through N standard Transformer encoder blocks — the *exact* mechanism from Chapter 7, with no vision-specific modification whatsoever. Multi-head self-attention lets every patch attend to every other patch, regardless of spatial distance — a sharp departure from CNNs, where a given layer can only see a local neighborhood (its receptive field, Deep Learning textbook Chapter 4.2), and only gradually accumulates a wider field of view across many stacked layers.

11.7 Step 6 — Classification Head

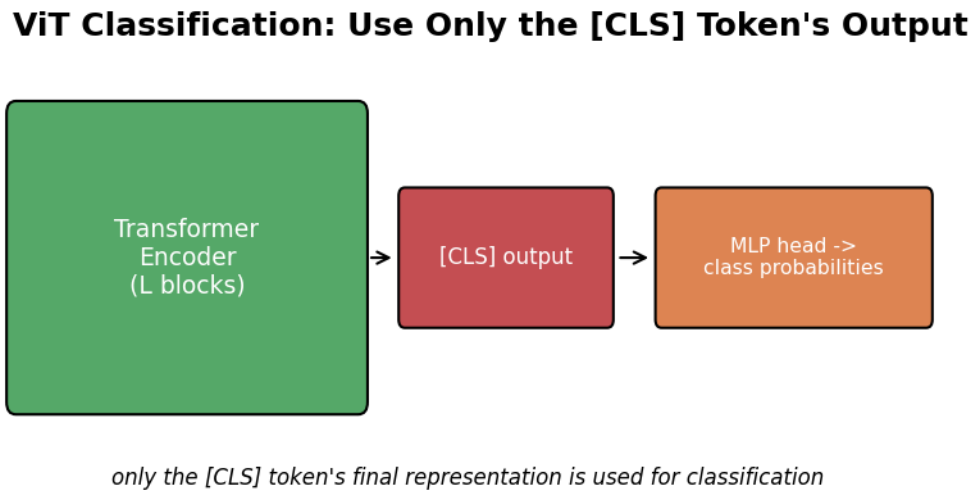


Figure 11: For classification, only the final [CLS] token's output representation is passed through a small MLP head to produce class probabilities — every other patch's final representation is simply discarded for this task (though it may well be used for other tasks, such as segmentation, that need per-patch output).

$$\hat{y} = \text{softmax}\big(W \cdot [\text{CLS}]_{\text{final}} + b\big)$$

11.8 Why This Actually Works — And Its One Real Weakness

CNNs have a strong **built-in inductive bias**: locality (nearby pixels matter more, by construction) and translation equivariance (a learned filter works the same everywhere in the image). ViT has *none* of this baked into its architecture — it has to *learn*, from data alone, that nearby patches tend to be more related, purely from the pressure of the training objective. This is precisely why the original ViT paper found that **ViT underperforms CNNs when trained on relatively small datasets**, but **matches or exceeds CNNs once trained on very large datasets** (hundreds of millions of images) — with enough data, the model learns useful spatial priors on its own, and the *lack* of a hard-coded bias becomes an advantage rather than a weakness, since the model isn't constrained to a human-designed assumption that might not perfectly fit the data.

11.9 Important ViT Variants

- **DeiT**: shows strong ViT performance is achievable on much smaller (ImageNet-scale, not "hundreds of millions of images"-scale) datasets, through a more careful training recipe and an additional **distillation token** — a second special token trained to mimic a pretrained CNN's predictions, injecting some of a CNN's useful bias back in via distillation rather than architecture.
- **Swin Transformer**: restricts self-attention to local, non-overlapping windows (reintroducing a form of locality bias, deliberately), then **shifts** the window boundaries between consecutive layers so that information can still eventually flow across the whole image over enough layers — dramatically cheaper for high-resolution inputs, and its hierarchical, multi-scale structure (progressively merging patches into larger ones at deeper layers) makes it much better suited than plain ViT to dense prediction tasks like object detection and segmentation, which need multi-scale feature maps.
- **MAE (Masked Autoencoders)**: a self-supervised pretraining scheme for ViT — mask out a large fraction (75%!) of image patches entirely, and train the model to reconstruct the missing pixels using only the visible patches. Extremely simple, no labels required, and produces excellent transferable features — vision's direct analogue of BERT's masked language modeling (Chapter 8).
- **DINO / DINOv2**: self-distillation training (a "student" and slowly-updated "teacher" version of the same network, trained to produce consistent representations for different augmented views of the same image) — produces general-purpose visual features, without any labels at all, that transfer strongly across a wide range of downstream tasks, and are widely used as a frozen, off-the-shelf visual backbone for other systems (including, notably, some robotics perception pipelines).

11.10 Where ViT Fits in the Bigger Picture of This Book

ViT is the single most important bridge in this entire book: it's the proof that the Transformer isn't a text-specific trick — it's a genuinely general architecture for *any* data that can be broken into a sequence of tokens, image patches included. This exact insight is what makes Chapter 12's multimodal models (which need vision and text to share a common representational language) and Chapter 15's robotics VLA models (which need vision, language, *and* robot actions to share one) architecturally possible in the first place.

12 Chapter 12 — Multimodal Transformers: Combining Vision and Language

12.1 Why This Is Now Straightforward, Given Everything Before This Chapter

Chapter 3 established the core trick that makes multimodal Transformers possible at all: **once anything is converted into a sequence of vectors of the same dimension d_{model}, a Transformer genuinely does not care what that data originally was.** Chapter 11 showed image patches can be embedded into that same space. This chapter is about what happens once you have both text tokens and image patches living in the same shared embedding space at once.

12.2 CLIP: Learning a Shared Space for Images and Text

CLIP (Contrastive Language-Image Pretraining) trains **two separate encoders** — a Vision Transformer (Chapter 11) for images, and a standard text Transformer (Chapters 2–7) for text — with one shared training objective: pull the embeddings of a matching image-text pair close together, and push non-matching pairs apart.

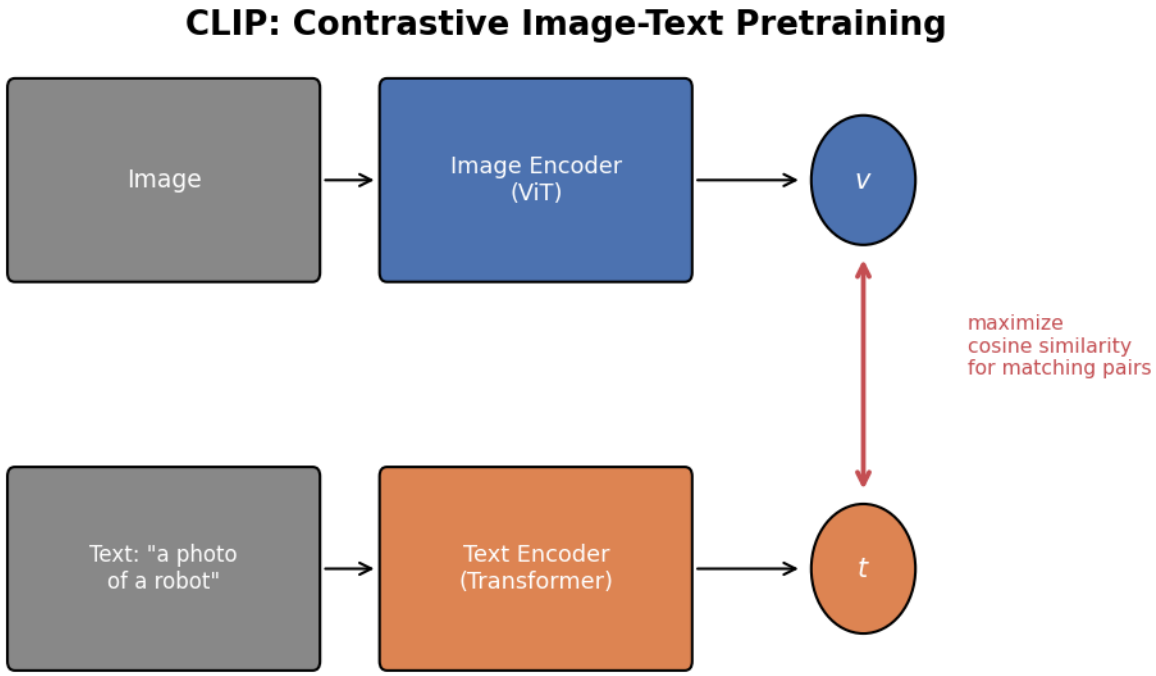


Figure 12: CLIP: an image encoder and a text encoder are trained jointly, with a contrastive objective that pulls matching image-text pairs' embeddings close together (high cosine similarity) and pushes non-matching pairs apart.

The training data: hundreds of millions of (image, caption) pairs scraped from the internet — no human labeling required beyond captions that already existed alongside images, making this a form of self-supervised learning at massive scale.

The loss, in plain terms: for a batch of N image-text pairs, compute the similarity between every image and every text in the batch ($N \times N$ similarity scores total). Train the model

so that each image's similarity to its *own* correct caption is high, and its similarity to every *other* caption in the batch is low — and symmetrically for text-to-image. This is the same **contrastive learning** principle behind self-supervised vision methods like SimCLR (Deep Learning textbook, Chapter 8.5), just applied across two different modalities instead of two augmented views of the same image.

Why CLIP became foundational: once trained, CLIP's image encoder produces embeddings that are already meaningfully aligned with language — enabling **zero-shot classification** (classify an image by comparing it against text embeddings of candidate class names, with no task-specific fine-tuning at all), and providing a ready-made, strong visual backbone that countless later systems build directly on top of.

12.3 Vision-Language Models (VLMs): From Alignment to Generation

CLIP aligns two embedding spaces but doesn't generate text. VLMs (BLIP, LLaVA, Flamingo, and similar) go a step further: they connect a vision encoder's output *directly into a large language model's input sequence*, so the LLM can generate free-form text conditioned on an image — captioning, visual question answering, or open-ended "chat about this image."

The typical recipe: take a pretrained vision encoder (often ViT-based, sometimes CLIP's own image encoder) and a pretrained decoder-only LLM (Chapter 8), and train a small **projection layer** that maps the vision encoder's patch embeddings into the *same dimensional space* the LLM's text token embeddings already live in. Once projected, the image's patch embeddings are simply **prepended to the text token sequence** and fed into the LLM like any other tokens — from the LLM's perspective, "image tokens" and "text tokens" are indistinguishable in *type*, only in *origin*. This is the single cleanest illustration of this chapter's opening claim: the Transformer doesn't know or care that some of its input tokens started life as pixels.

12.4 Why This Design Is So Reusable

This exact pattern — pretrain strong single-modality encoders separately, then train a comparatively small and cheap projection/fusion layer to align them into a shared token space — is now the standard blueprint for multimodal systems generally, and, critically for your own work, it's *exactly* the blueprint Chapter 15's robotics Vision-Language-Action models reuse, with one further extension: a third modality (robot actions) gets folded into the same shared sequence alongside vision and language tokens.

13 Chapter 13 — Efficient Attention: Scaling to Long Sequences

13.1 The Problem, Stated Precisely

Chapter 5's attention computes an $n \times n$ matrix of scores, where n is sequence length — every token compared against every other token. This means both the compute *and* memory cost of standard attention scale as $O(n^2)$. Double the sequence length, and you quadruple the cost. For a modest sequence of a few hundred tokens this is a non-issue; for a long document, a long conversation history, a high-resolution image split into many patches (Chapter 11), or a long robot trajectory (relevant to Chapter 15), this quadratic cost becomes the dominant practical bottleneck in the entire system.

13.2 FlashAttention: Same Math, Dramatically Faster

FlashAttention doesn't change *what* attention computes — it's mathematically exact, not an approximation — it changes *how* the computation is organized on the GPU. Standard implementations materialize the full $n \times n$ score matrix in the GPU's slow, high-capacity memory (HBM); FlashAttention instead processes attention in small blocks that fit entirely within the GPU's much faster, much smaller on-chip memory (SRAM), fusing the score computation, softmax, and value-weighting into one pass without ever writing the full $n \times n$ matrix out to slow memory at all. The result is a substantial wall-clock speedup and reduced memory usage, purely from being *IO-aware* — this has become close to a standard default in modern Transformer training and inference stacks, essentially a free efficiency win with no accuracy trade-off.

13.3 Sparse and Windowed Attention: Genuinely Changing What Gets Computed

Unlike FlashAttention, the following approaches *do* trade off some modeling capacity for speed, by simply not computing every pairwise attention score at all:

- **Sliding window attention**: each token only attends to a fixed-size neighborhood of nearby tokens (e.g. the 512 tokens immediately before and after it), reducing cost to $O(n \cdot w)$ where w is the window size — genuinely linear in sequence length. The obvious cost: a token can no longer directly attend to something far outside its window in a single layer (though information can still propagate across longer ranges indirectly, over multiple stacked layers, similar in spirit to how a CNN's receptive field grows with depth).
- **Sparse/strided attention** (Longformer, BigBird): combine a small local window with a sparse set of longer-range connections (e.g. every token also attends to a handful of fixed "global" positions, or to every k-th token) — an attempt to keep some long-range modeling capacity while still achieving much better than quadratic scaling.

13.4 Linear Attention Approximations

A different family of approaches (Linformer, Performer, and similar) reformulates the attention computation mathematically to avoid ever forming the full $n \times n$ matrix, achieving $O(n)$ scaling — usually by low-rank approximating the score matrix, or by reformulating softmax attention as a kernel function that can be computed in a different, cheaper order. These methods trade some exactness for asymptotic scaling, and in practice see mixed adoption — many production systems have found FlashAttention (exact, just faster) combined with sliding-window/sparse patterns for very long contexts to be a more practically reliable combination than fully linear approximations.

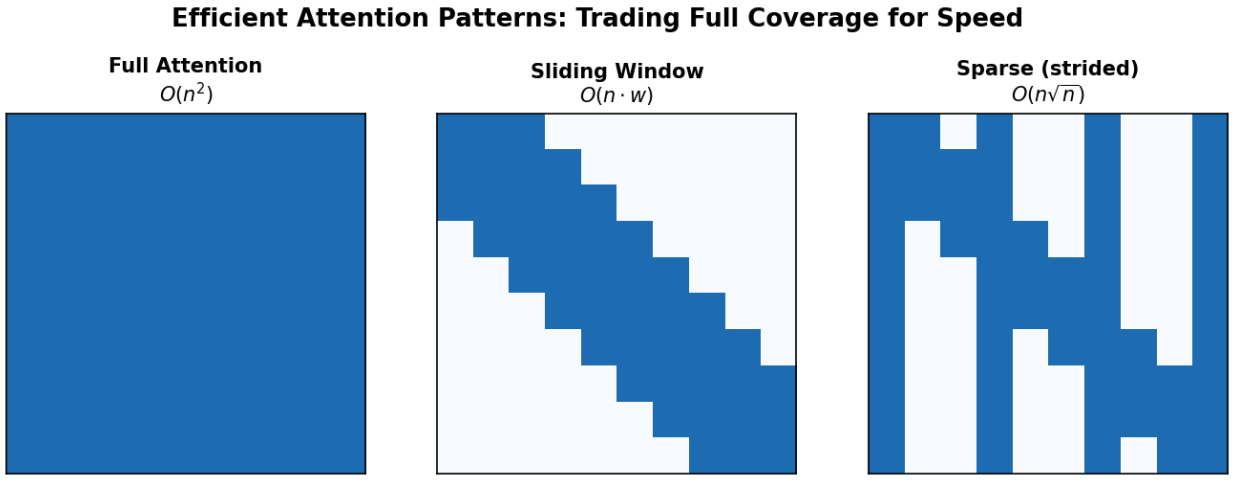


Figure 13: Three attention patterns compared: full attention computes every pair (quadratic cost); sliding-window attention only computes scores within a local neighborhood around each token (linear cost, local context only); sparse/strided attention mixes a small local window with periodic long-range connections, approximating global reach at a fraction of full attention's cost.

13.5 Mixture of Experts (MoE): A Different Axis of Efficiency Entirely

Worth distinguishing clearly from everything above: MoE (Deep Learning textbook, Chapter 6.8) doesn't make *attention* cheaper — it makes the *feed-forward sublayer* (Chapter 7) cheaper relative to total parameter count, by routing each token to only a small subset of many available "expert" sub-networks rather than running every token through one dense feed-forward layer. This is a genuinely separate efficiency lever from everything else in this chapter, and the two are frequently combined in the same large-scale model — efficient attention to handle long sequences cheaply, and MoE to scale total parameter count without a proportional increase in per-token compute.

13.6 A Practical Framing

None of these techniques change *what* a Transformer fundamentally is — every one of them is still built from Chapters 2–7's core ingredients. They're better understood as **engineering answers to a single, very concrete constraint**: standard attention's $O(n^2)$ cost becomes the bottleneck once sequences get long enough, and this chapter's techniques are the accumulated toolbox for pushing that bottleneck back, each with a different trade-off between exactness, long-range modeling capacity, and raw speed.

14 Chapter 14 — Where Transformers Are Used: An Industry Survey

14.1 Natural Language Processing (The Original Home Turf)

Machine translation, summarization, question answering, sentiment analysis, chatbots and virtual assistants, code generation and completion (Chapter 15's robotics discussion aside, this is itself a massive industrial application — modern coding assistants are decoder-only Transformers trained heavily on code), and search/retrieval (encoder-only models producing embeddings used for semantic search, frequently paired with a **RAG** pipeline — retrieving relevant documents via embedding similarity, then feeding them into a generative model's context, a pattern covered in the Agentic AI material).

14.2 Computer Vision

Image classification, object detection and segmentation (Swin-Transformer-based backbones are now common in top-performing detection/segmentation systems), image generation (Vision Transformers are used as backbones inside modern diffusion models — see the Deep Learning textbook's generative models chapter — and some diffusion architectures, like Diffusion Transformers/DiT, replace the traditional U-Net backbone with a Transformer entirely), video understanding (video Transformers extend ViT's patch idea across the time dimension as well as space), and medical imaging (Transformer-based models for tumor detection, radiology report generation, and similar diagnostic-support tasks).

14.3 Speech and Audio

Speech recognition (Whisper and similar models use an encoder-decoder Transformer directly on audio spectrograms), text-to-speech and voice synthesis, and music generation.

14.4 Science and Biology

AlphaFold 2, arguably one of the most consequential scientific applications of this architecture family, uses Transformer-style attention mechanisms (a specialized variant, adapted to protein structure) to predict 3D protein structure from amino acid sequence — a problem that had resisted decades of prior computational approaches, addressed effectively once framed as a sequence-relationship problem attention is naturally suited to. Similar sequence-modeling approaches are increasingly applied to genomics, drug discovery (molecule generation and property prediction), and materials science.

14.5 Finance and Time Series

Transformer-based models for forecasting (stock prices, demand forecasting, anomaly detection in transaction data), where the same long-range dependency modeling that helps with long documents helps with long historical time series — though it's worth noting that, for many time-series tasks specifically, well-tuned classical or simpler deep learning approaches remain highly competitive, and Transformers' advantage here is less universally dominant than in language or vision.

14.6 Recommendation Systems

Modeling a user's sequence of past interactions (clicks, purchases, watched videos) as a token sequence, and using a Transformer (typically decoder-only, next-item-prediction-style) to predict what they're likely to engage with next — an increasingly common architecture

at large-scale consumer platforms, replacing older sequence models (including RNN-based ones) for the same reasons covered throughout this book.

14.7 Enterprise and Productivity Software

Document understanding and information extraction, customer support automation, code review and static analysis assistance, and increasingly, **agentic systems** (covered in the Agentic AI material) that use a decoder-only LLM as the reasoning core of a larger system that also calls external tools, searches the web, and takes multi-step actions toward a goal — the Transformer here functions as the "brain" inside a considerably larger engineered system.

14.8 And, the Focus of the Next Chapter — Robotics

Everything above treats a Transformer as processing and generating symbols — words, image patches, or audio frames. Chapter 15 covers what happens when a Transformer's output is extended one step further: not just symbols *about* the world, but direct commands *to act on* the world.

15 Chapter 15 — Transformers in Robotics and Embodied AI

15.1 The Core Idea: Treat a Robot Action as Just Another Token

Every chapter up to this point has shown the same pattern repeat across modalities: text becomes tokens (Chapter 2), images become patch tokens (Chapter 11), and a shared embedding space lets a single Transformer reason over multiple modalities at once (Chapter 12). Robotics extends this pattern one final step, and it's a genuinely elegant idea: **discretize a robot's continuous actions (joint angles, gripper open/close, end-effector position deltas) into tokens too**, and let the *same* decoder-only Transformer architecture from Chapter 8 predict the next action token exactly the way it would predict the next word.

15.2 Vision-Language-Action (VLA) Models

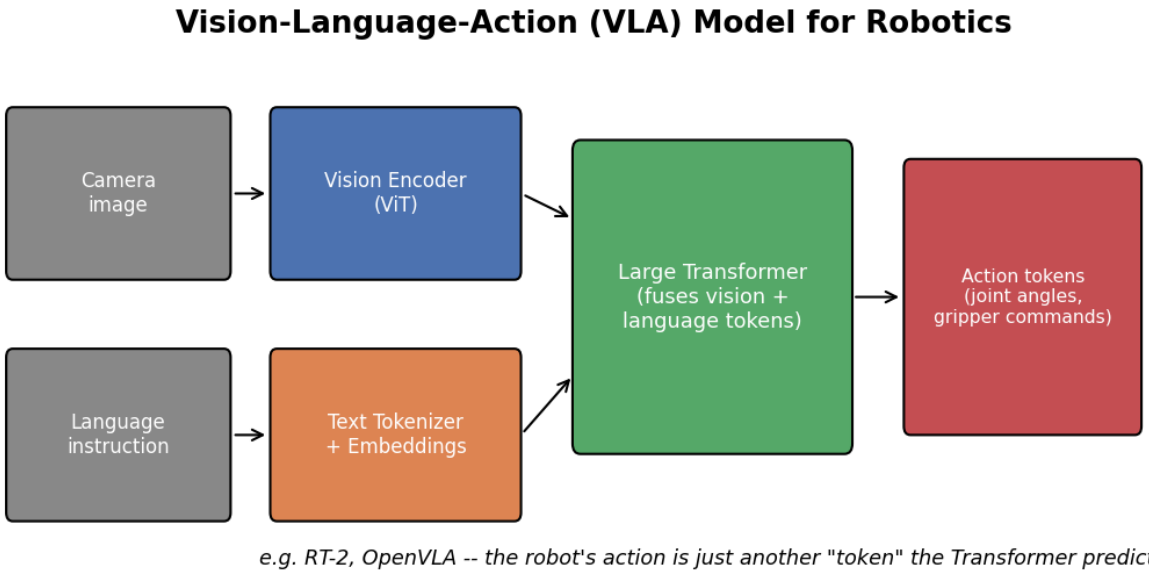


Figure 14: A Vision-Language-Action model: a camera image and a language instruction are both encoded into the same token space (via a ViT and a text tokenizer, exactly as in Chapter 12), fused inside a large Transformer, which outputs action tokens — discretized joint angles or gripper commands — read directly by the robot's controller.

The pipeline, end to end:

1. **Vision input**: a camera frame is patch-embedded via a Vision Transformer (Chapter 11) — often initialized from a large pretrained vision-language model like CLIP (Chapter 12), rather than trained from scratch, to inherit strong general visual understanding before ever seeing a single robot demonstration.
2. **Language input**: a natural-language instruction ("pick up the red block and place it in the bin") is tokenized (Chapter 2) exactly like any text input to an LLM.
3. **Fusion**: image tokens and text tokens are combined into one sequence (Chapter 12's exact recipe) and processed by a large decoder-only Transformer.
4. **Action output**: instead of predicting the next *word*, the model predicts the next **action**

token — a robot's continuous joint angles and gripper state are discretized (binned into, e.g., 256 possible values per degree of freedom) so they can be represented as tokens from a fixed vocabulary, exactly like Chapter 2's text vocabulary, just repurposed for motor commands.

Why discretize continuous actions into tokens at all, rather than just having the network output real numbers directly? Because it lets the *entire* existing Transformer machinery — the same causal decoder architecture, the same next-token training objective, the same autoregressive generation loop — apply completely unmodified. A robot action becomes "just another word" in an unusually shaped vocabulary, and the model doesn't need any specialized regression head or different training procedure at all.

15.3 Concrete Named Systems

- **RT-1 (Robotics Transformer 1)**: one of the first large-scale demonstrations that a single Transformer policy, trained on a large, diverse dataset of real robot demonstrations, could generalize across many manipulation tasks and object types far better than smaller task-specific policies.
- **RT-2**: extends RT-1 by initializing from a large pretrained Vision-Language Model (rather than training vision understanding from scratch on robot data alone) and co-training on both web-scale vision-language data *and* robot action data simultaneously. The striking result: the model exhibits meaningfully better generalization to novel objects and instructions it never saw in robot-specific training — because it inherited broad visual/semantic understanding from the much larger, much more diverse web-scale pretraining, in exactly the same "pretrain broad, fine-tune narrow" spirit as Chapter 9.
- **OpenVLA**: an openly released Vision-Language-Action model, trained on a large aggregated cross-embodiment dataset (real demonstrations spanning many different robot platforms, not just one), directly relevant to the general principle (also covered in the RL+IL book, Chapter 14) that training on diverse embodiments improves generalization even when you only care about deploying on one specific robot.
- **Decision Transformer**: a distinct and conceptually interesting approach — instead of the VLA pattern above, it reframes reinforcement learning itself (RL+IL book, Chapters 1–11) as pure sequence modeling: feed in a sequence of past states, actions, and a **target return** (how much total reward you want to achieve), and the model predicts the next action needed to achieve that return, using nothing but a standard causal Transformer decoder with no separate value function, policy gradient, or Bellman equation in sight — RL's entire problem, reframed as "predict the next token in a sequence that happens to end well."

15.4 How This Connects to Imitation Learning (RL+IL Book, Chapters 12–15)

VLA models are, from a training-objective standpoint, almost always trained via **Behavior Cloning** (RL+IL book, Chapter 12) at the largest possible scale: enormous datasets of (camera image, language instruction, expert action) tuples, with a next-action-token prediction loss — mechanically identical to Chapter 9's next-token prediction loss in this very book, just applied to action tokens instead of word tokens. The RL+IL book's Chapter 14 discussion of **Diffusion Policy** and **Action Chunking (ACT)** are worth revisiting here with fresh eyes: Action Chunking, in particular, is directly motivated by a Transformer-specific insight — predicting a short *sequence* of several future action tokens at once (rather than one token at a time) reduces the compounding-error problem (RL+IL book, Chapter 12) in exactly the way the RL+IL book describes, and fits naturally into a Transformer's native ability to predict multiple future positions in one pass.

15.5 Why This Matters for ROS 2-Based Systems Specifically

A trained VLA model, once deployed, is typically wrapped as a ROS 2 node: it subscribes to a `sensor_msgs/Image` topic (camera feed) and a topic or service carrying the language instruction, and publishes to whatever topic drives the robot's low-level controller (e.g. a `FollowJointTrajectory` action goal, or a `geometry_msgs/Twist` for mobile base velocity commands) — sitting at exactly the perception-to-planning boundary the ROS 2 material in the earlier AI Engineering guide described. The Transformer itself has no notion of ROS 2, topics, or the control stack underneath it — from its perspective, it's simply predicting the next token in a sequence, precisely as this entire book has described, with a robot's actuators as the very last, very literal consumer of that prediction.

16 Chapter 16 — Implementing a Transformer in PyTorch

Every block below maps directly to a chapter earlier in this book — the variable names are chosen to match the formulas exactly, so you can trace any line of code back to the equation it implements.

16.1 Tokenization (Chapter 2) — Using a Real Library

Implementing BPE from scratch is a genuinely useful exercise, but in practice you almost always use an existing, pretrained tokenizer:

```
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
tokens = tokenizer.tokenize("Robots learn quickly")
ids = tokenizer.convert_tokens_to_ids(tokens)
print(tokens)   # ['robots', 'learn', 'quickly']
print(ids)      # [8123, 4553, 2855]  (actual IDs depend on the vocabulary)
```

16.2 Embeddings (Chapter 3)

```
import torch
import torch.nn as nn

vocab_size, d_model = 30000, 512
token_embedding = nn.Embedding(vocab_size, d_model)
x = token_embedding(torch.tensor(ids))   # shape: (seq_len, d_model)
```

16.3 Positional Encoding (Chapter 4) — Sinusoidal, From Scratch

```
import math

class SinusoidalPositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=5000):
        super().__init__()
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2) * (-math.log(10000.0) / d_model))
        pe[:, 0::2] = torch.sin(position * div_term)   # even dims: sine
        pe[:, 1::2] = torch.cos(position * div_term)   # odd dims: cosine
        self.register_buffer('pe', pe)

    def forward(self, x):
        # x shape: (batch, seq_len, d_model)
        return x + self.pe[:x.size(1)]
```

16.4 Scaled Dot-Product Self-Attention (Chapter 5) — From Scratch

```
def scaled_dot_product_attention(Q, K, V, mask=None):
    d_k = Q.size(-1)
    scores = Q @ K.transpose(-2, -1) / math.sqrt(d_k)   # QK^T / sqrt(d_k)
    if mask is not None:
```

```
        scores = scores.masked_fill(mask == 0, float('-inf'))  # causal mask, Chapter 8
    weights = torch.softmax(scores, dim=-1)
    return weights @ V, weights
```

16.5 Multi-Head Attention (Chapter 6) — From Scratch

```
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, num_heads):
        super().__init__()
        assert d_model % num_heads == 0
        self.d_k = d_model // num_heads
        self.num_heads = num_heads
        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)
        self.W_o = nn.Linear(d_model, d_model)

    def forward(self, x, mask=None):
        batch, seq_len, d_model = x.shape
        Q = self.W_q(x).view(batch, seq_len, self.num_heads, self.d_k).transpose(1, 2)
        K = self.W_k(x).view(batch, seq_len, self.num_heads, self.d_k).transpose(1, 2)
        V = self.W_v(x).view(batch, seq_len, self.num_heads, self.d_k).transpose(1, 2)
        out, _ = scaled_dot_product_attention(Q, K, V, mask)
        out = out.transpose(1, 2).contiguous().view(batch, seq_len, d_model)
        return self.W_o(out)   # final linear projection, Chapter 6
```

Or, using PyTorch's built-in, production-ready implementation:

```
mha = nn.MultiheadAttention(embed_dim=512, num_heads=8, batch_first=True)
out, attn_weights = mha(query=x, key=x, value=x)   # self-attention: Q, K, V all from x
```

16.6 A Full Transformer Encoder Block (Chapter 7) — From Scratch

```
class TransformerBlock(nn.Module):
    def __init__(self, d_model, num_heads, d_ff):
        super().__init__()
        self.attn = MultiHeadAttention(d_model, num_heads)
        self.norm1 = nn.LayerNorm(d_model)
        self.ff = nn.Sequential(
            nn.Linear(d_model, d_ff),
            nn.GELU(),
            nn.Linear(d_ff, d_model),
        )
        self.norm2 = nn.LayerNorm(d_model)

    def forward(self, x, mask=None):
        x = self.norm1(x + self.attn(x, mask))     # sublayer 1 + residual + norm
        x = self.norm2(x + self.ff(x))              # sublayer 2 + residual + norm
        return x

# Stacking N blocks, per Chapter 7's closing note
encoder = nn.ModuleList([TransformerBlock(d_model=512, num_heads=8, d_ff=2048) for _ in range(6
```

16.7 Causal Masking for a Decoder-Only Model (Chapter 8)

```
def causal_mask(seq_len):
    # lower-triangular matrix: 1 = allowed to attend, 0 = masked out
    return torch.tril(torch.ones(seq_len, seq_len)).unsqueeze(0).unsqueeze(0)

mask = causal_mask(seq_len=6)
out = decoder_block(x, mask=mask)   # each position can only see itself and earlier positions
```

16.8 A Minimal Vision Transformer Patch Embedding (Chapter 11)

```
class PatchEmbedding(nn.Module):
    def __init__(self, img_size=224, patch_size=16, in_channels=3, d_model=768):
        super().__init__()
        self.num_patches = (img_size // patch_size) ** 2
        # a single conv with kernel=stride=patch_size does patch-extraction + linear projection
        self.proj = nn.Conv2d(in_channels, d_model, kernel_size=patch_size, stride=patch_size)
        self.cls_token = nn.Parameter(torch.zeros(1, 1, d_model))
        self.pos_embedding = nn.Parameter(torch.zeros(1, self.num_patches + 1, d_model))

    def forward(self, img):
        batch = img.shape[0]
        x = self.proj(img)                          # (batch, d_model, H/patch, W/patch)
        x = x.flatten(2).transpose(1, 2)            # (batch, num_patches, d_model)
        cls_tokens = self.cls_token.expand(batch, -1, -1)
        x = torch.cat([cls_tokens, x], dim=1)       # prepend [CLS], Chapter 11 Step 3
        return x + self.pos_embedding               # add positional embeddings, Step 4
```

16.9 Using a Pretrained Transformer Directly (What You'll Actually Do Most Often)

```
from transformers import AutoModel, ViTModel

text_model = AutoModel.from_pretrained("bert-base-uncased")
vision_model = ViTModel.from_pretrained("google/vit-base-patch16-224")

# text_model / vision_model already implement everything in Chapters 2-11 correctly and efficie
# in practice, you fine-tune these rather than reimplementing them from scratch for production
```

16.10 Closing Note

Just as with the RNN book's code chapter, the point of tracing every equation into runnable code isn't to memorize these snippets — it's to close the loop between "I understand the formula" and "I could actually build this." If any line above is unclear, the fix is returning to that specific chapter's derivation, not treating the code as a separate thing to learn independently.